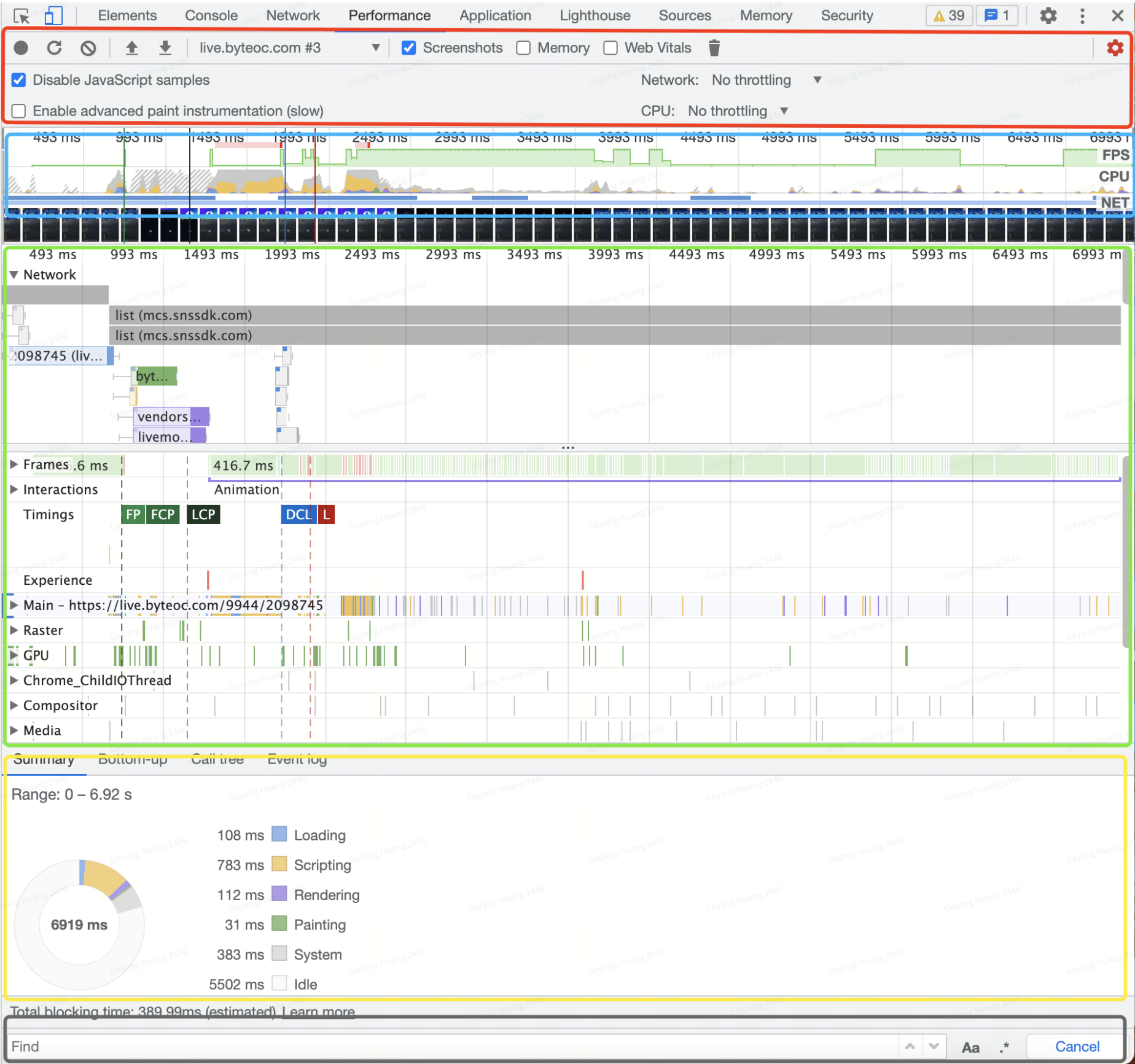


Chrome DevTools Performance 功能详解

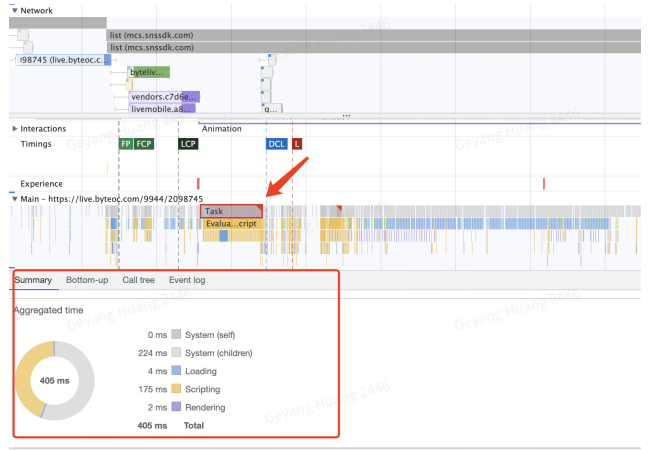
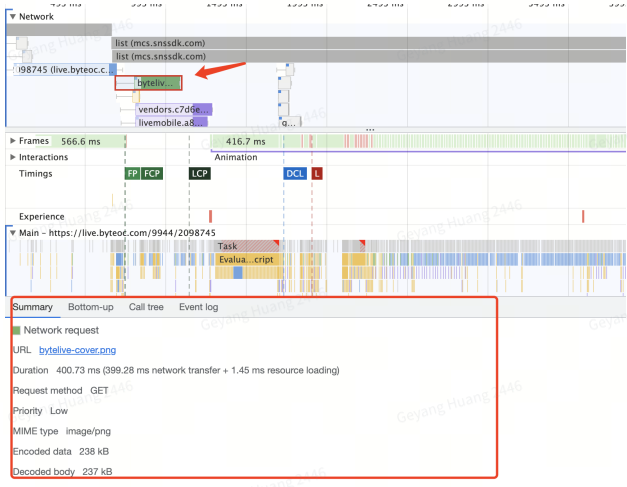
简介

本文介绍 Chrome DevTools Performance 面板的所有功能，以便使用其分析页面加载时/运行时性能，找出性能瓶颈。

功能面板概览

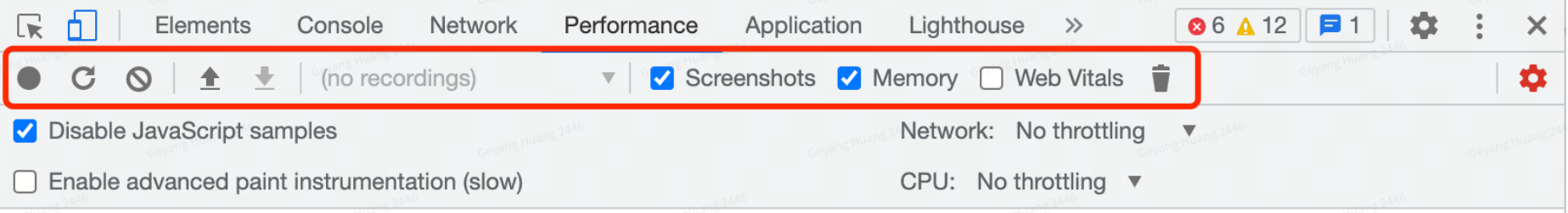


- 控制面板（红色区域）：控制性能分析相关功能的配置。
- 概览面板（蓝色区域）：主要性能项目的图形化预览。可选择录制的某一个片段，默认为完整的录制片段。
- 视图面板（绿色区域）：展示 概览面板 中选择的片段的其他指标数据。点击此面板中的内容可进行选择。
- 详情面板（黄色区域）：展示 视图面板 中选择内容的详情。其中 Summary 选项卡为点选内容的数据总览，不同的点击项目展示的条目有所区别。而 Bottom-up、Call tree、Event log 选项卡主要与主线程活动相关，会在 Main 中做详细介绍。



- 搜索框（黑色区域）：按 Command+F (Mac) 或 Control+F（Windows、Linux）打开底部的搜索框，可对 Main 中火焰图中的活动进行搜索。

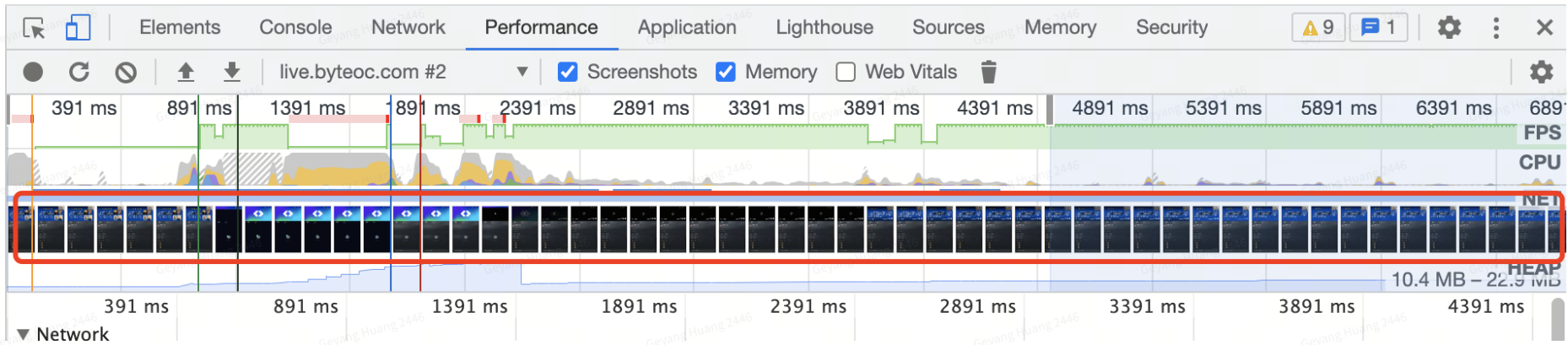
控制面板



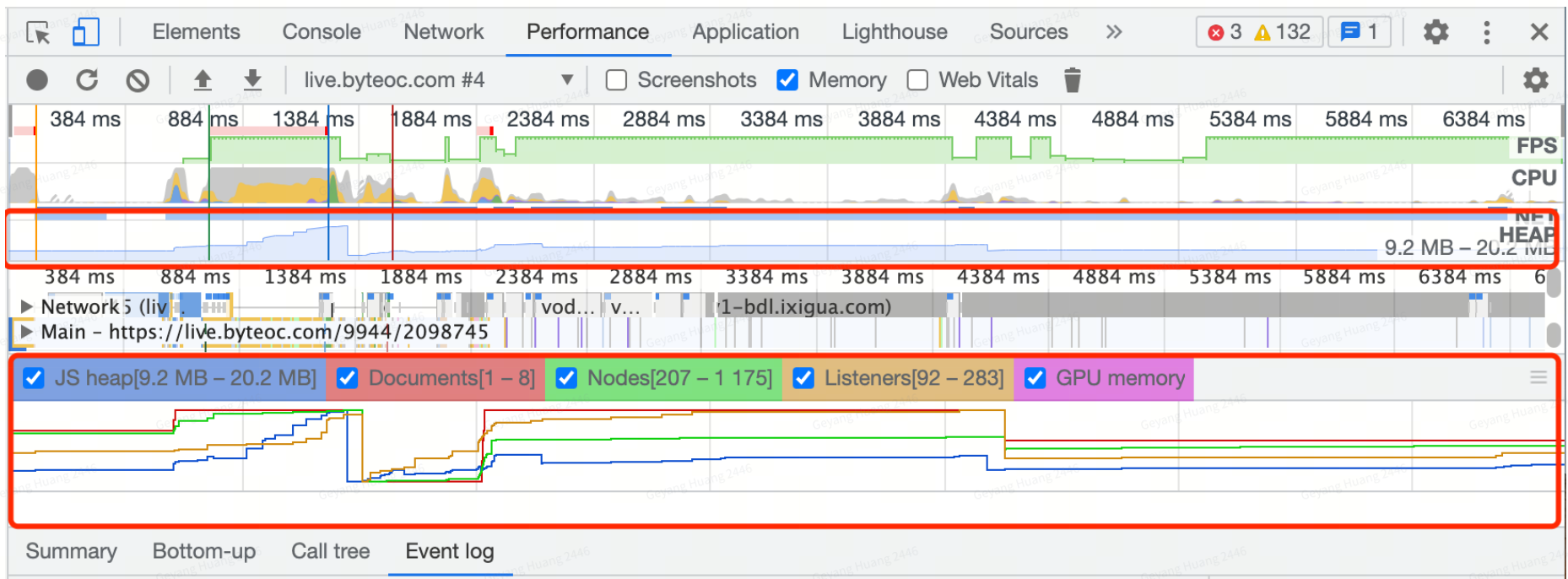
- 红色区域从左至右的功能分别为
- Record：开始/停止记录页面运行时性能，再次点击结束记录并生成分析报告。
 - Start profiling and load page：重新加载页面并记录页面加载时的性能，页面加载完成后会自动停止记录。
 - Clear：清空所有录制的分析内容。
 - Load profile：上传之前保存的分析报告。
 - Save profile：将当前记录的分析内容以 JSON 文件形式保存。每次记录都会生成一个性能分析报告可供下载；也可截取记录中的某段内容生成分析报告进行保存。
 - Show recent timeline sessions：选择最近的性能分析记录进行显示。



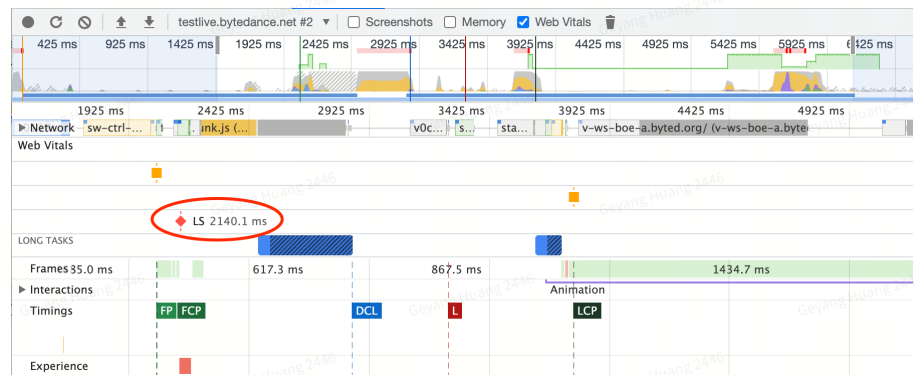
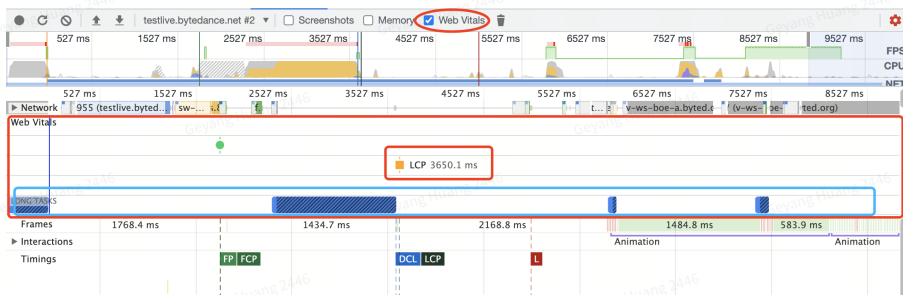
- Screenshots：是否启用屏幕截图。启用后将会在录制时捕获每一帧的屏幕截图。



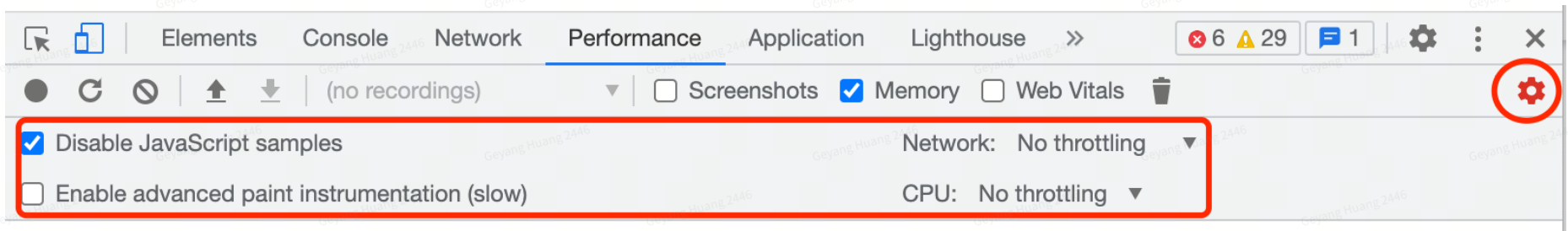
- **Memory**：是否显示内存指标。勾选后会展示一个内存线形图，并且 **NET** 图表下方会展示一个 **HEAP** 图。HEAP 图与内存图中 **JS Heap** 的信息相同，表示JS堆内存，内存飙升可能意味着**内存泄漏**，内存不足又可能引发页面崩溃。浏览器在回收内存时还会暂停执行JS，从而使得页面因为GC（垃圾回收）而出现卡顿或频繁暂停现象。而 **JS heap** 选项右边分别是文档、DOM 节点、监听器和 GPU 内存的内存使用情况，这些内存使用变化可能与JS执行存在相关性（比如某个事件执行注入了大量的节点）。



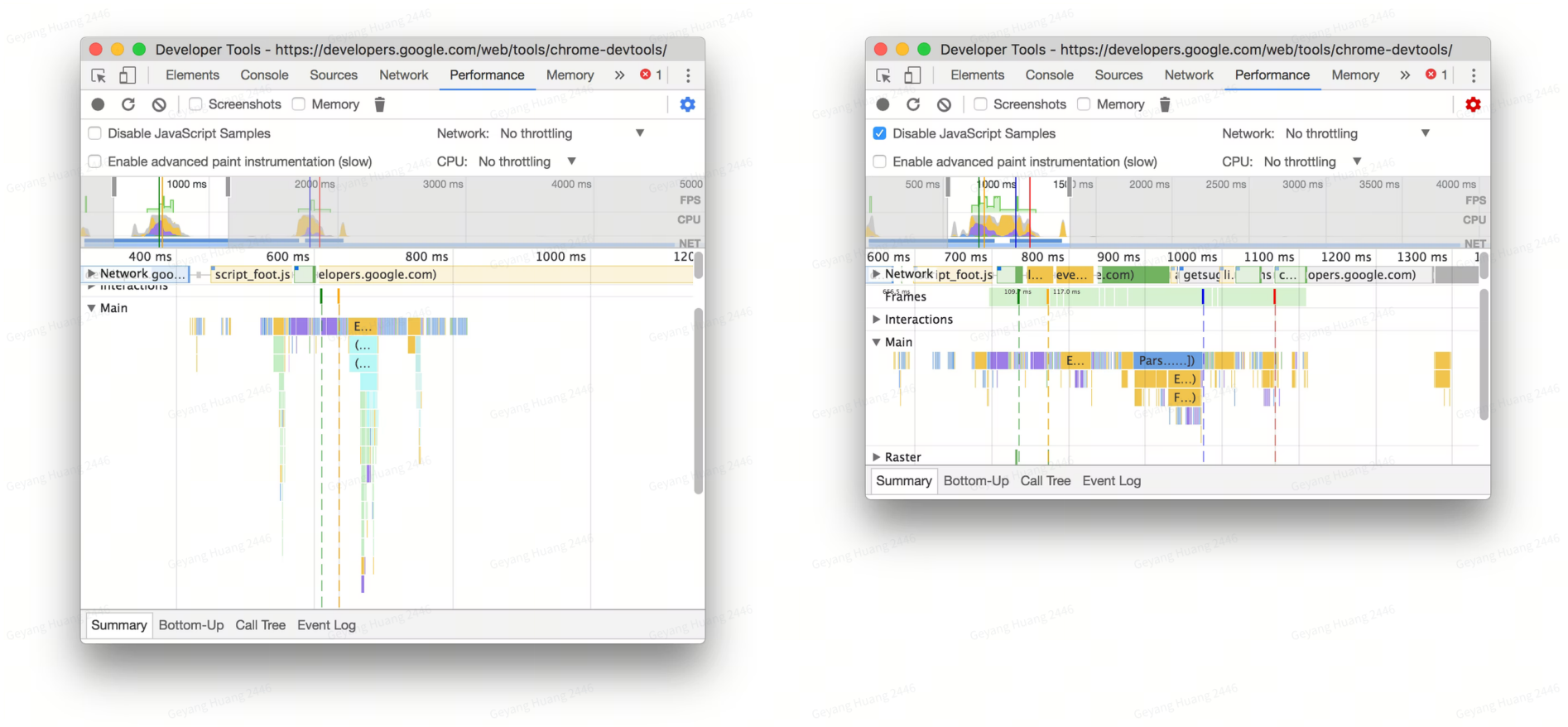
- **Web Vitals**：展示能够最快提升当前网页性能表现的核心指标和影响因素，类似一个简易的 **Lighthouse**。绿色圆点表示健康表现的指标，黄色小方形、红色菱形则分别中等、较差表现得的指标，如下方左图的 LCP、右图中的 LS（Layout shift）；蓝色矩形表示导致页面卡顿、掉帧的长任务（超过50ms）。



- **Collect rubbish**：点击其强制进行垃圾收集。



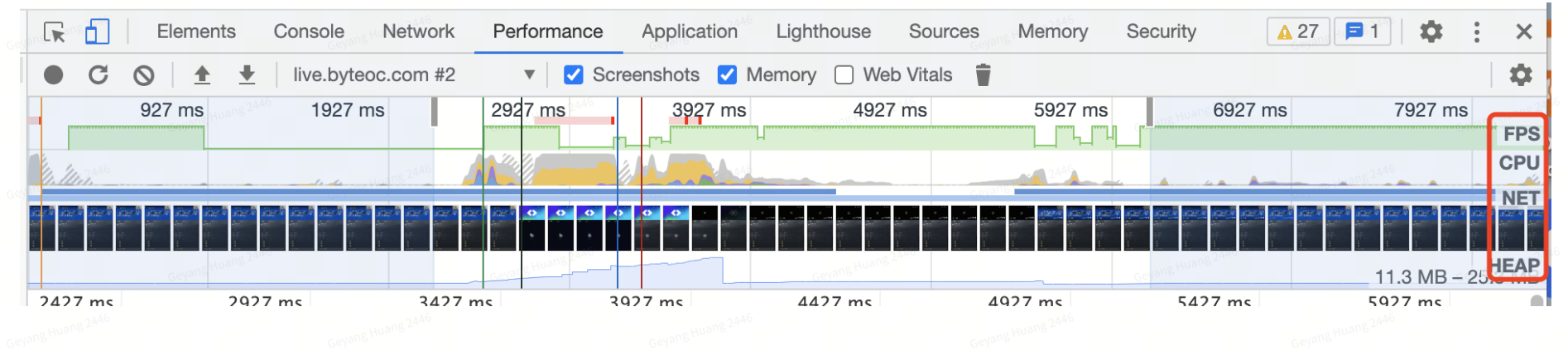
- **Disable JavaScript samples**：是否隐藏调用堆栈。默认情况下，**Main** 中会详细记录 JavaScript 的调用堆栈（左图）。



- Network: 设置网络状况。用于模拟弱网或离线网络状态。
- Enable advanced paint instrumentation(slow): 启用高级绘画工具。用于开启一些高级分析功能，比如[图层信息](#)、[Paint 分析器](#)等。
- CPU: 设置 CPU 能力。常用于模拟移动设备 CPU，由于移动设备的 CPU 能力远低于 PC，若设置 `4x slowdown` 则表示 DevTools 会限制当前 CPU 使其比现在慢 4 倍。

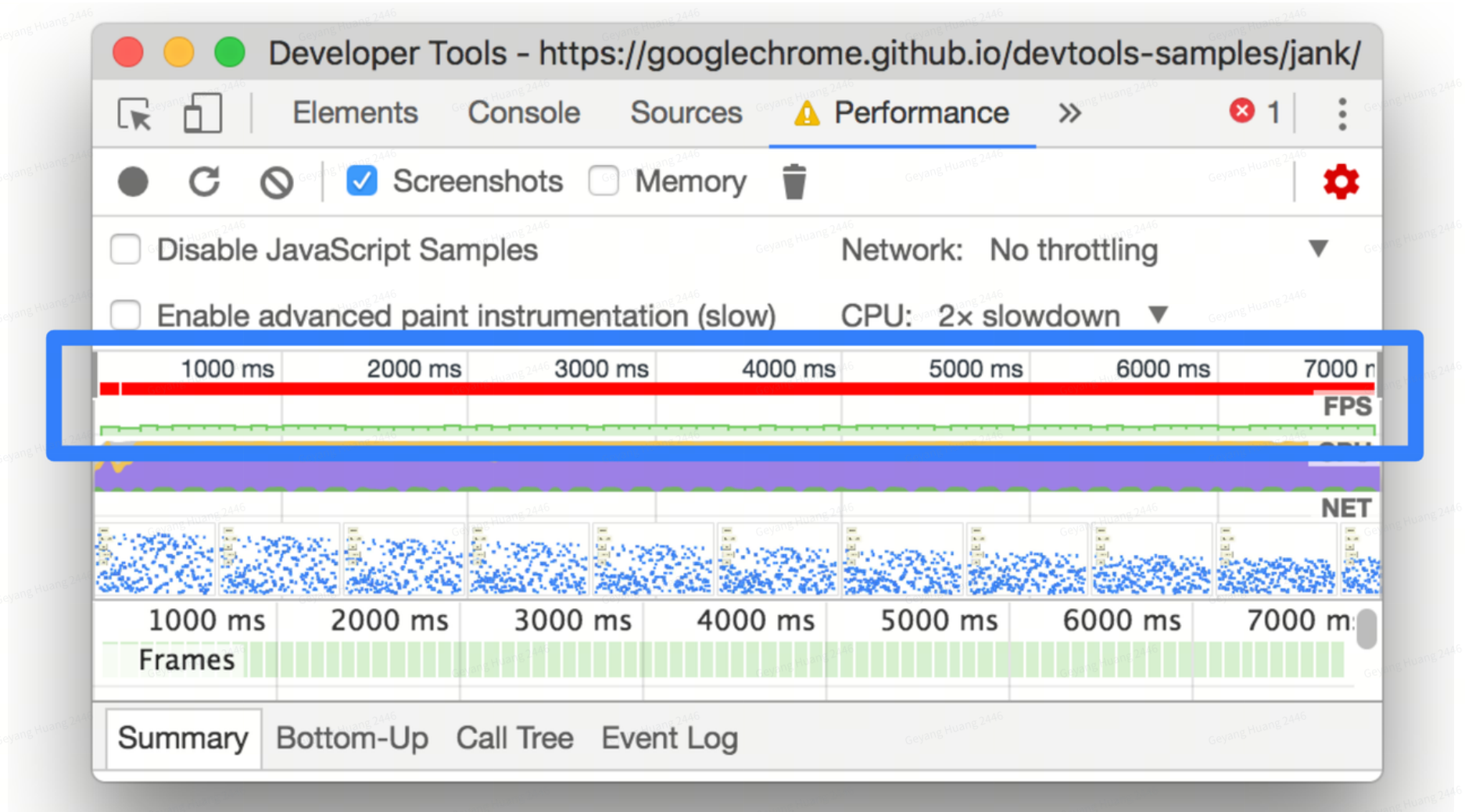
概览面板

性能面板为FPS，CPU和NET图表数据的概览，在概览上向左或向右拖动鼠标、或点击某区域可选择其中一部分录制内容。



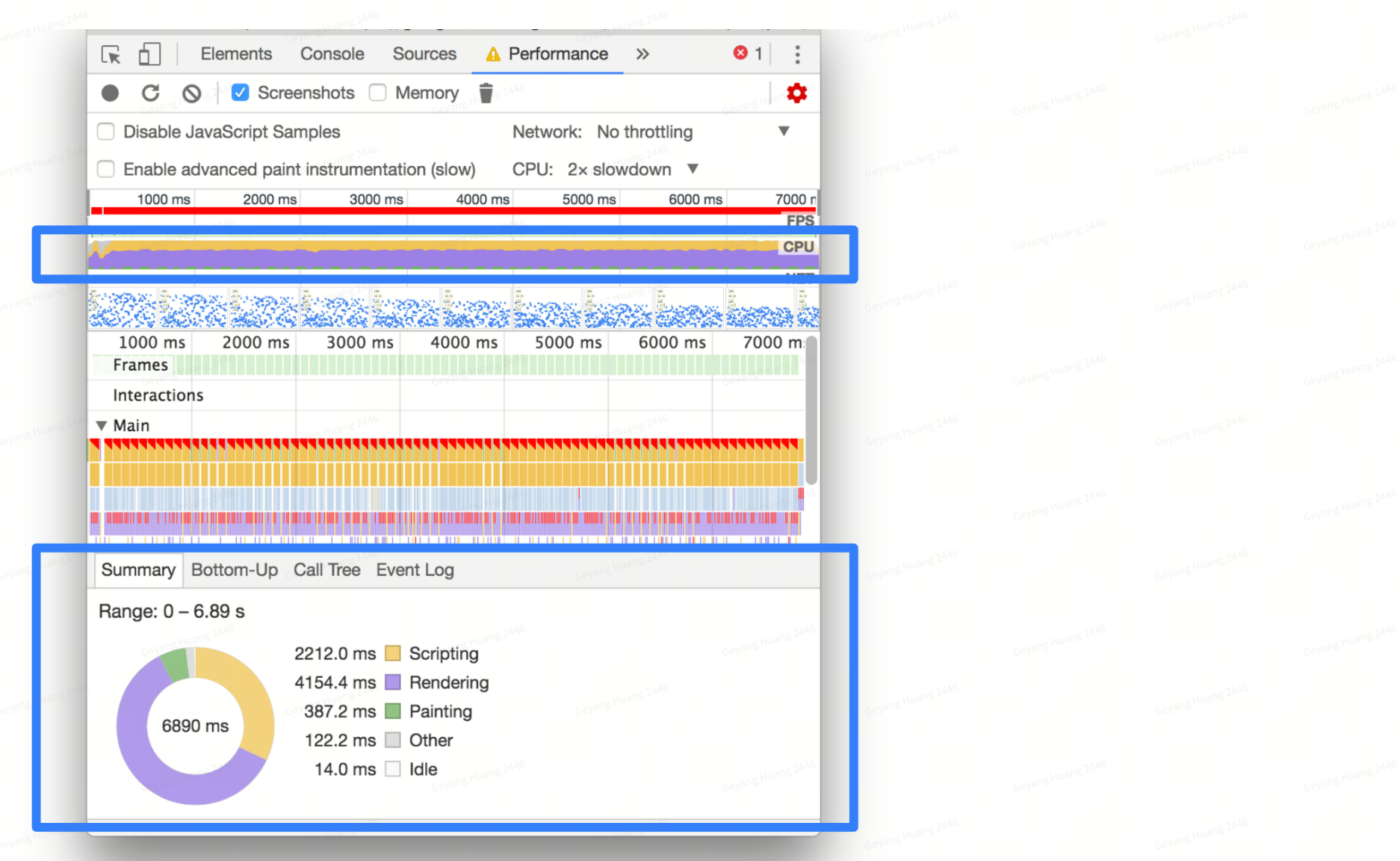
FPS

每秒帧数。通常情况下 FPS 越高越好，当 $FPS \geq 60$ 时页面刷新频率能与大多数浏览器的刷新频率（60Hz）相吻合。`FPS` 上方出现红色条时就意味着出现长时间帧（帧率下降），从而出现页面卡顿有损用户体验；而绿色条越高则表示 FPS 越高，用户体验越好。



CPU

CPU 资源使用情况。CPU 中面积图如果充满色彩就表示该时间段 CPU 已达到极限。当看到 CPU 长时间处于最大状态时就是需要寻找减少 CPU 工作量方法的提示。



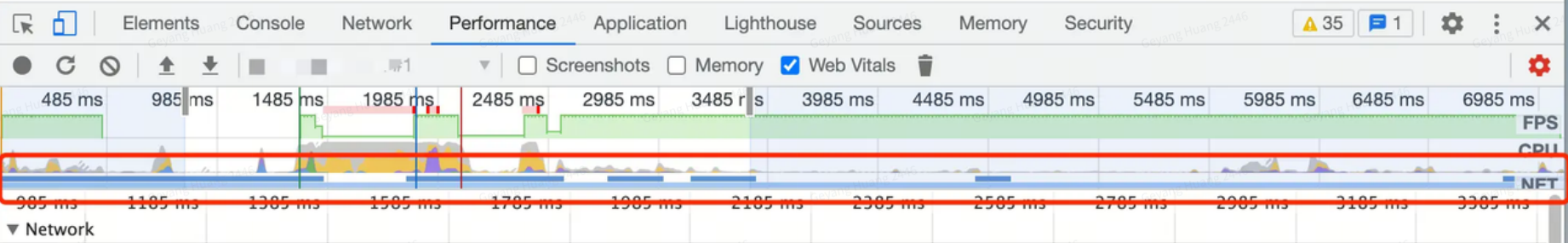
不同颜色的面积图表示不同的消耗 CPU 资源的事件类型，并且面积图中的各种颜色与 Summary 中的颜色相对应：

- 蓝色>Loading): 表示网络通信和 HTML 解析时间。
- 黄色>Scripting): 表示 JavaScript 执行时间。
- 紫色>Rendering): 表示样式计算和布局（重排）时间。

- 绿色(Painting)：表示重绘时间。
- 灰色(other)：表示其它事件花费的时间。
- 白色(Idle)：表示空闲时间。

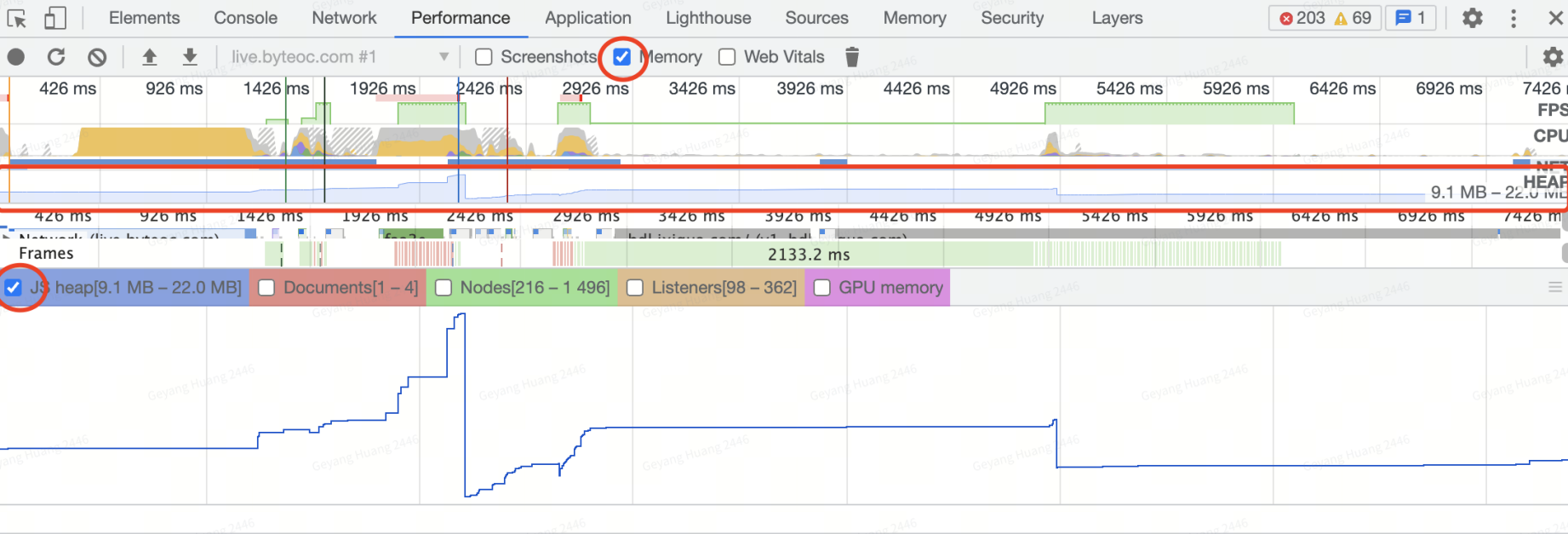
NET

Network 概览。深蓝色表示存在高优先级的资源请求的时间段，浅蓝色表示存在低优先级的资源请求的时间段。在 [Network](#) 部分进行详解。



HEAP

JS heap 使用情况。与开启 `Memory` 后的 JS heap 线形图一致。

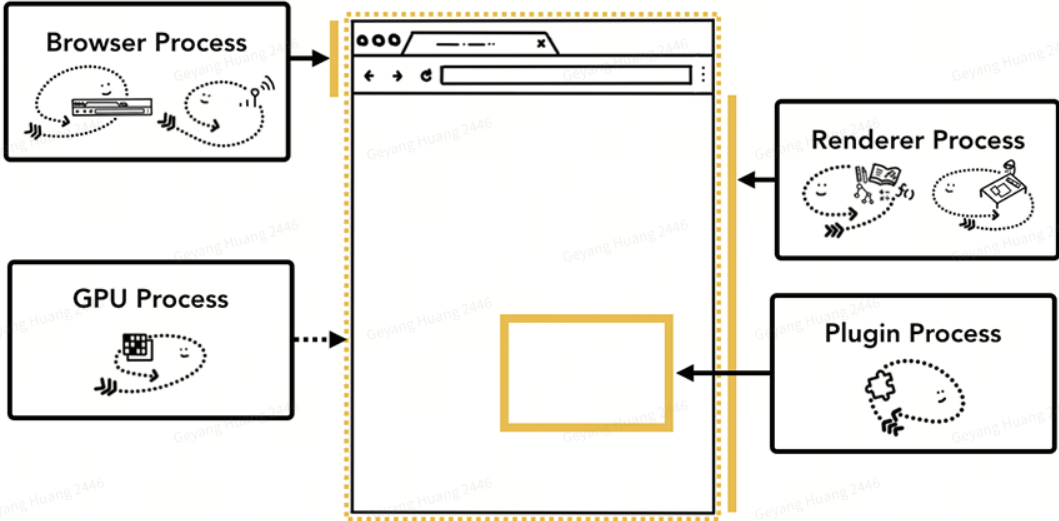


Renderer Process

视图面板中的功能使用基本都很简单，但理解有一定成本，因此为了更好的介绍视图面板中 Frames、Raster、Main等功能，先对渲染器进程做一下赘叙。也可以直接跳过此内容至[视图面板](#)。

Chrome 的多进程架构中，每打开一个页签都会独立运行一个渲染器进程，从而保证某一页签无响应或崩溃不会影响其他页签的活动。

Chrome 中的进程与分工：

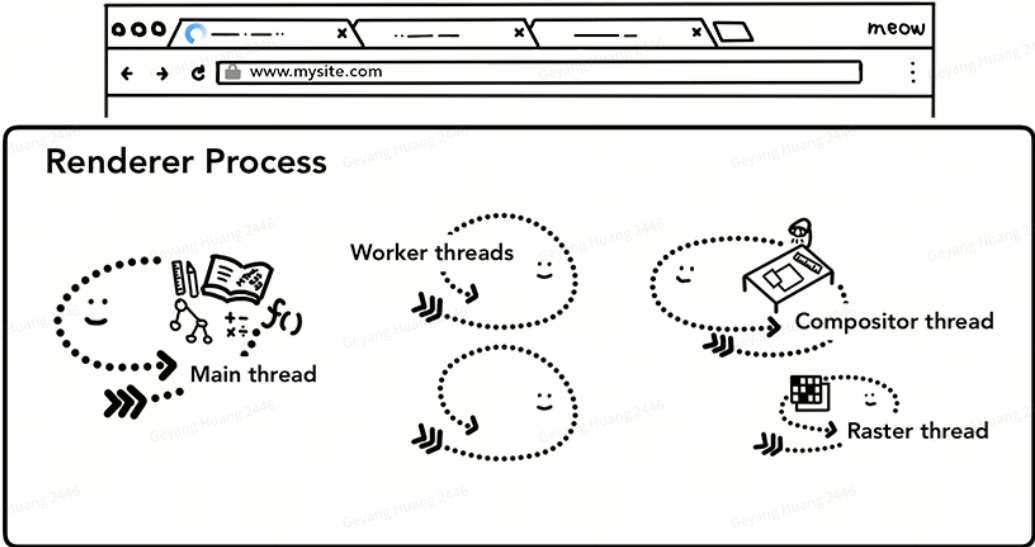


- 浏览器进程（Browser Process）：控制 chrome 应用程序，包括地址栏、书签、后退和前进按钮。还处理 Web 浏览器的不可见的特权部分，例如网络请求和文件访问。

- 渲染器进程（Renderer Process）：控制页签中所有内容的显示。
- 插件进程（Plugin Process）：控制网站使用的任何插件，例如 flash。
- 图形处理器进程（GPU Process）：独立于其他进程处理 GPU 任务。

渲染器进程中的线程：

渲染器进程包含主线程（main thread）、工作线程（worker threads）、合成线程（compositor thread）和光栅线程（raster thread）。



页面渲染步骤

渲染器进程的核心工作是将 HTML、CSS 和 JavaScript 转换为用户可以与之交互的网页，主要步骤如下：

1. DOM 解析（Parsing）

渲染进程解析接收到的HTML数据并转化为DOM对象。

2. 样式计算（Style calculation）

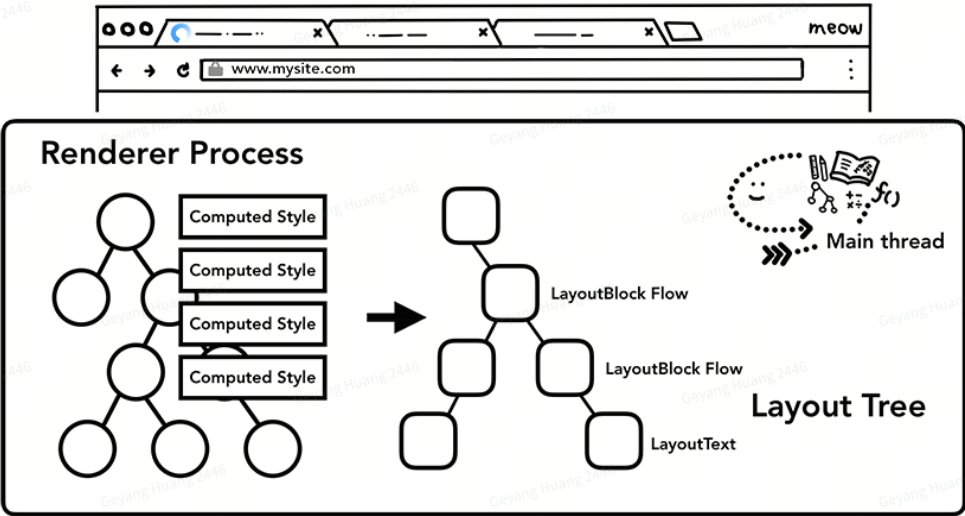
主线程根据 CSS 样式选择器计算出的每个DOM元素应具备的具体样式。

3. 布局（Layout）

经过上面的步骤虽然已经知道页面的具体文档结构以及每个节点拥有的样式信息，但仍然不能确定页面最终的样子。布局的过程是计算出每个节点的几何信息。



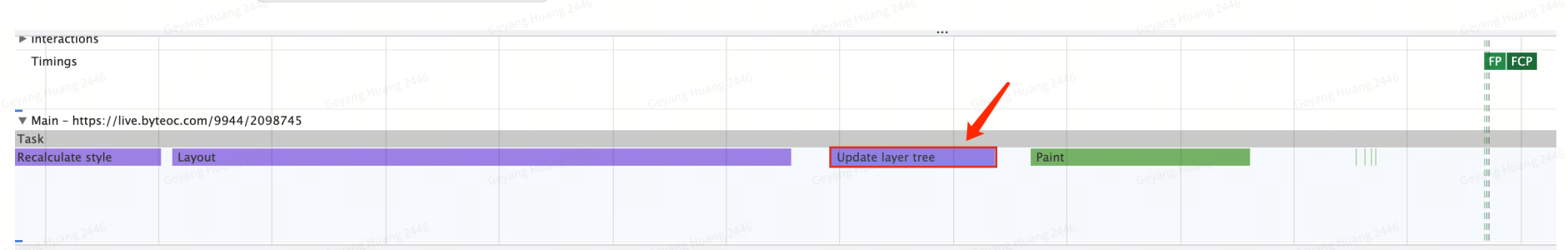
布局过程中主线程会遍历构建的DOM树，根据DOM节点的计算样式计算出一个布局树（layout tree）。



布局树上每个节点会有它在页面上的x, y坐标以及盒子大小（bounding box sizes）的具体信息。布局树与先前构建的DOM树差不多，不同的是只有那些可见的节点。

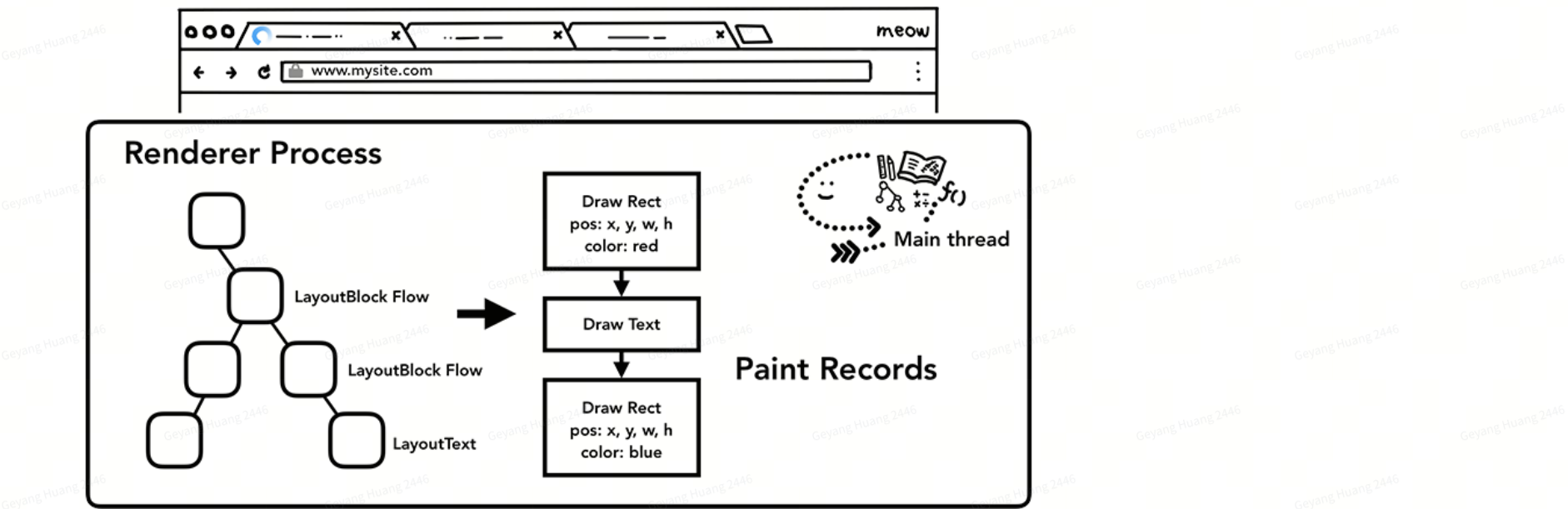
4. 分层（Dividing into layers）

分层的作用确定哪些元素需要放置在哪一图层，此过程主线程会遍历布局树来创建一棵层次树（Layer Tree）。在DevTools中这一部分工作叫做 `Update Layer Tree`。分层与合成（Compositing）息息相关，在[后面](#)详细介绍。



5. 绘制 (Paint)

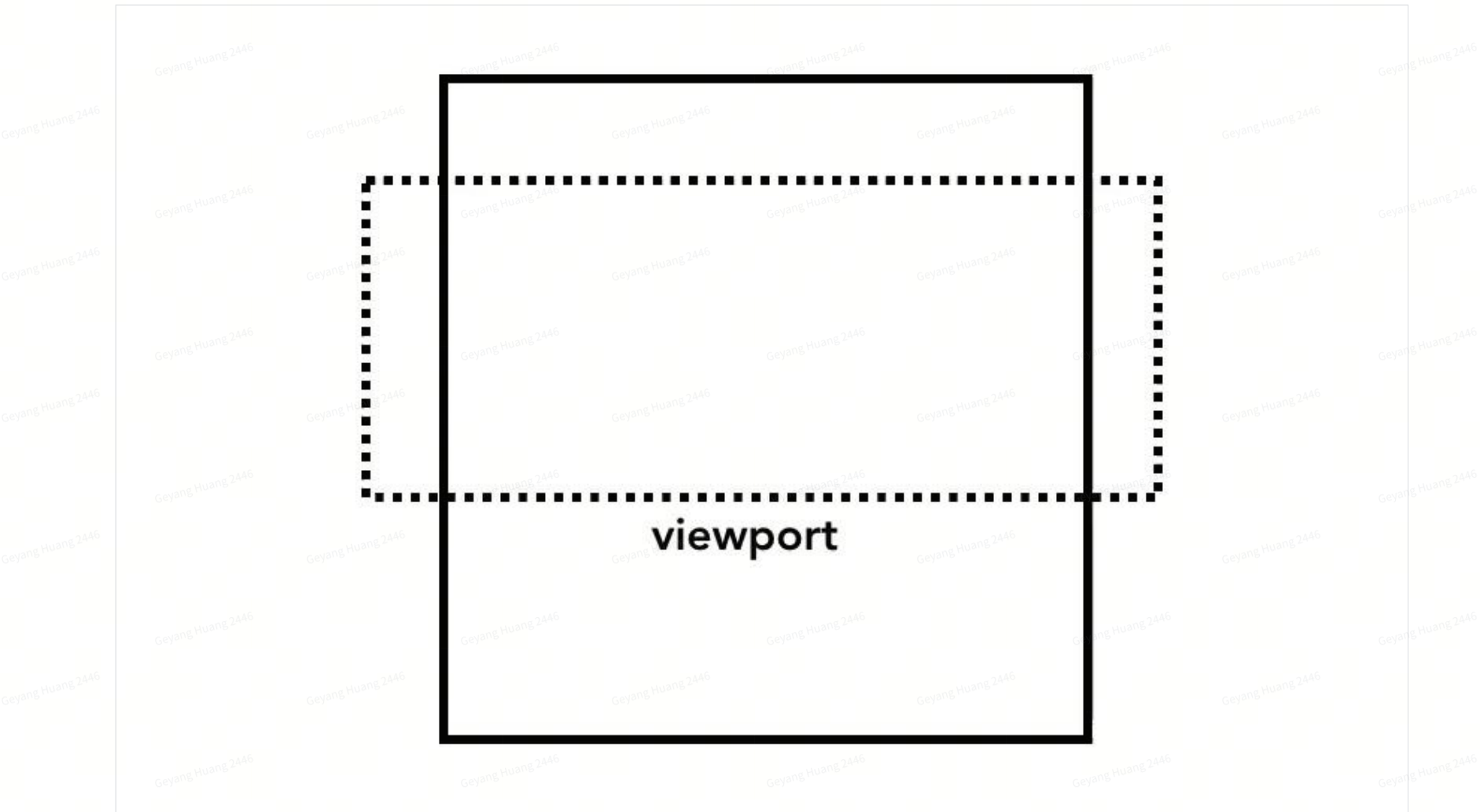
有了 DOM、样式和布局信息仍不足以呈现页面。比如某些元素设置了 `z-index`，仅按照 HTML 元素的顺序会导致渲染的结果不正确，所以还需要知道绘制的顺序。主线程会遍历之前的布局树（layout tree）来生成一系列的绘画记录（paint records），从而得到了绘制的顺序。



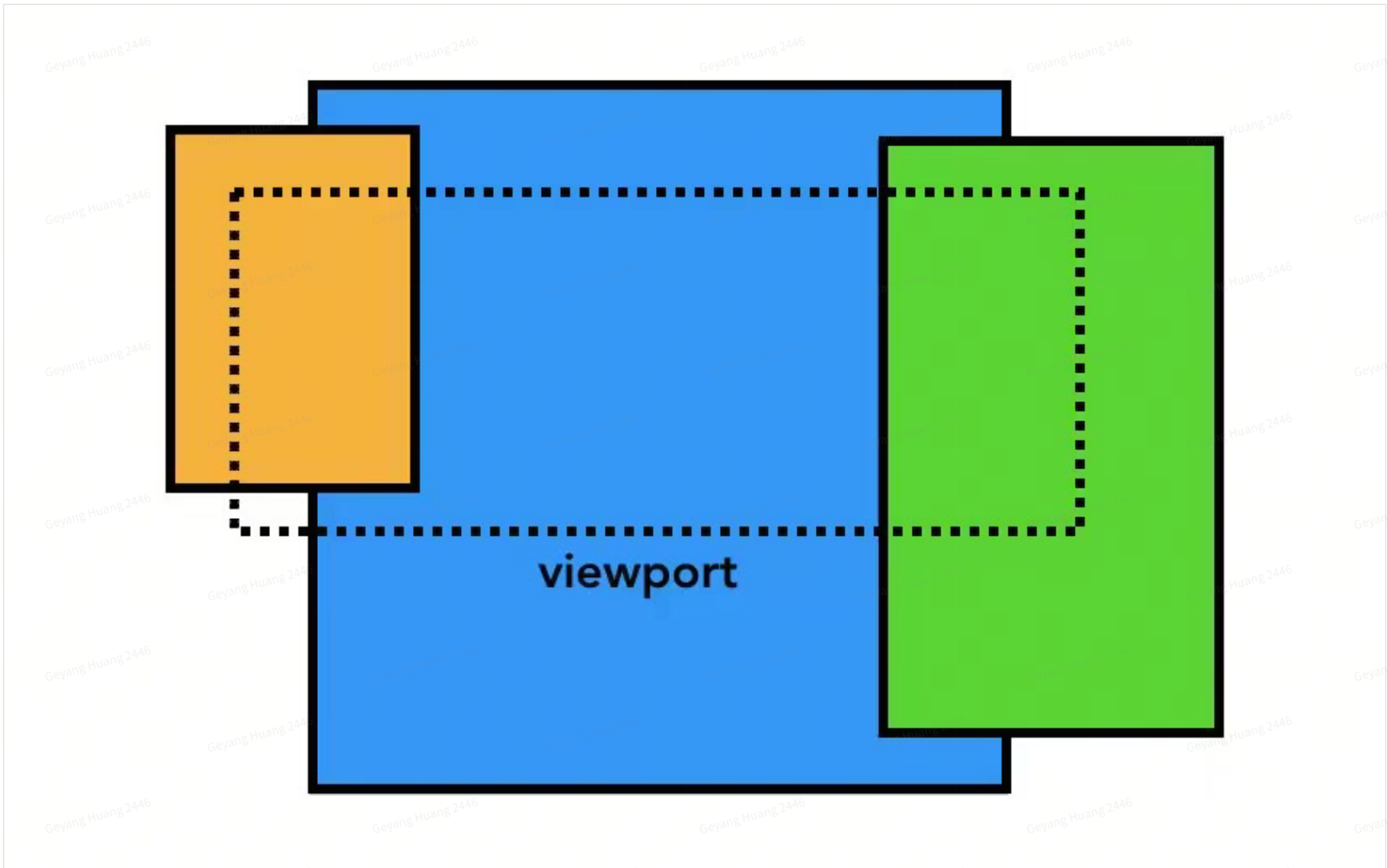
6. 合成（Compositing）

经过上面的步骤，浏览器知道了文档结构、每个元素的样式、页面的几何形状和绘制顺序。浏览器就可以开始绘制页面。而将这些信息转化为显示器的像素的过程叫做**光栅化**。

光栅化最简单的做法就是只光栅化视口内的内容。如果用户进行了页面滚动，就移动光栅帧（rastered frame）光栅化更多的内容以填充缺失的部分👉，最初的 Chrome 就是这么做的。



对于现代的浏览器来说采取一种更加复杂的做法，叫做合成（compositing）。合成是一种将页面分成若干层，分别对它们进行光栅化，然后在一个单独的线程 — 合成线程（compositor thread）里合成为一个页面的技术。当用户滚动页面时，由于页面各个层都已经被光栅化了，浏览器需要做的只是合成一个新的帧来展示滚动后的效果👉。动画则可以通过移动图层并合成新帧以相同的方式实现。

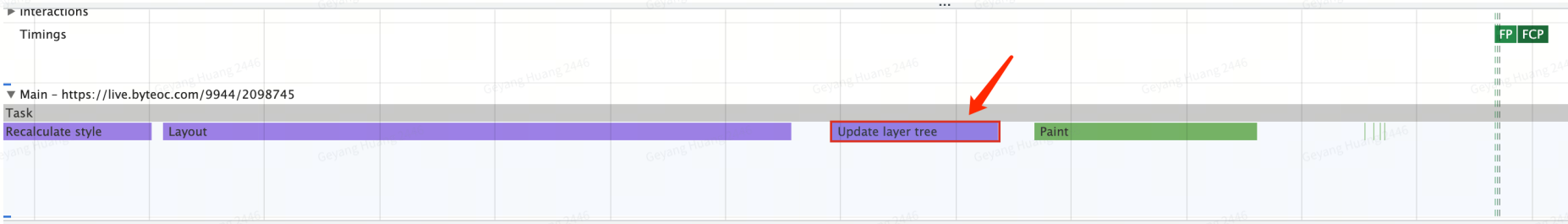


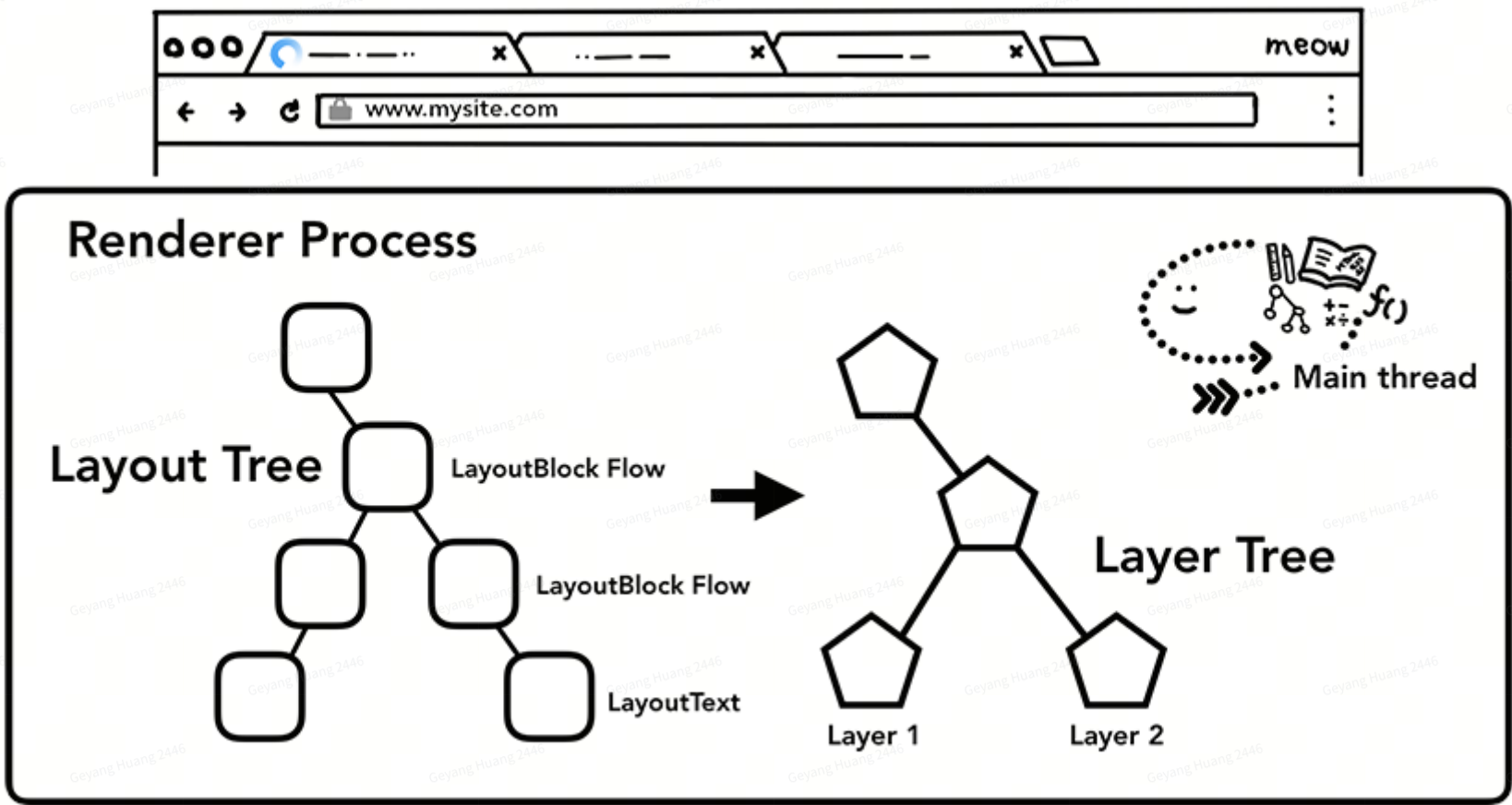
光栅化与合成

合成的过程类似 Photoshop 将多个图层按顺序叠加后导出最终的图片，图像发生变更也只需要调整对应的图层。合成的详细过程：

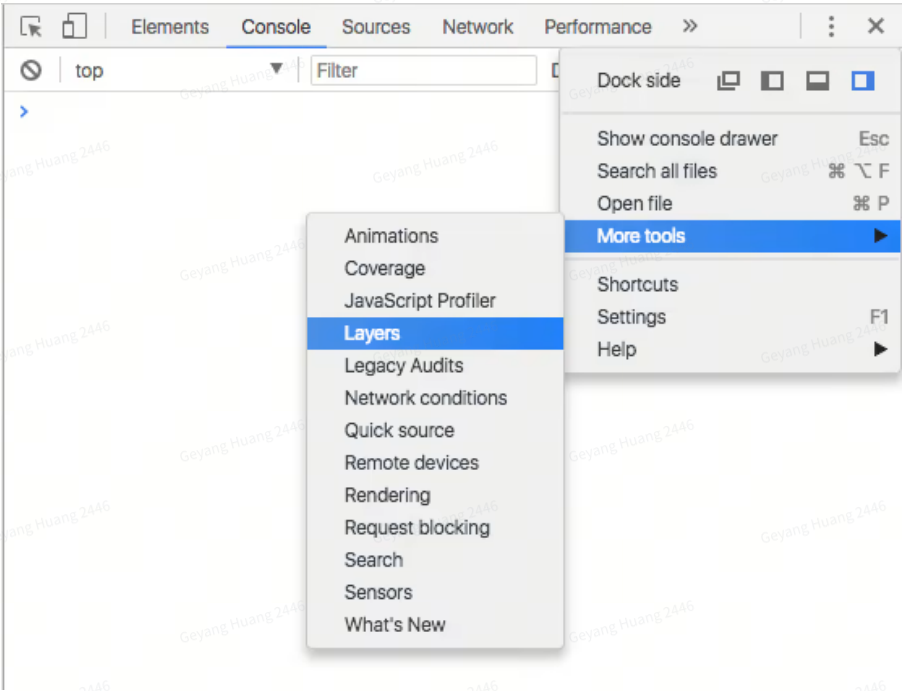
1. 分层（Dividing into layers）

合成之前需要主线程先将页面分成若干层。为了确定哪些元素需要放置在哪一层，主线程需要遍历布局树来创建一棵层次树（Layer Tree），在DevTools中这一部分工作叫做 `Update Layer Tree`。



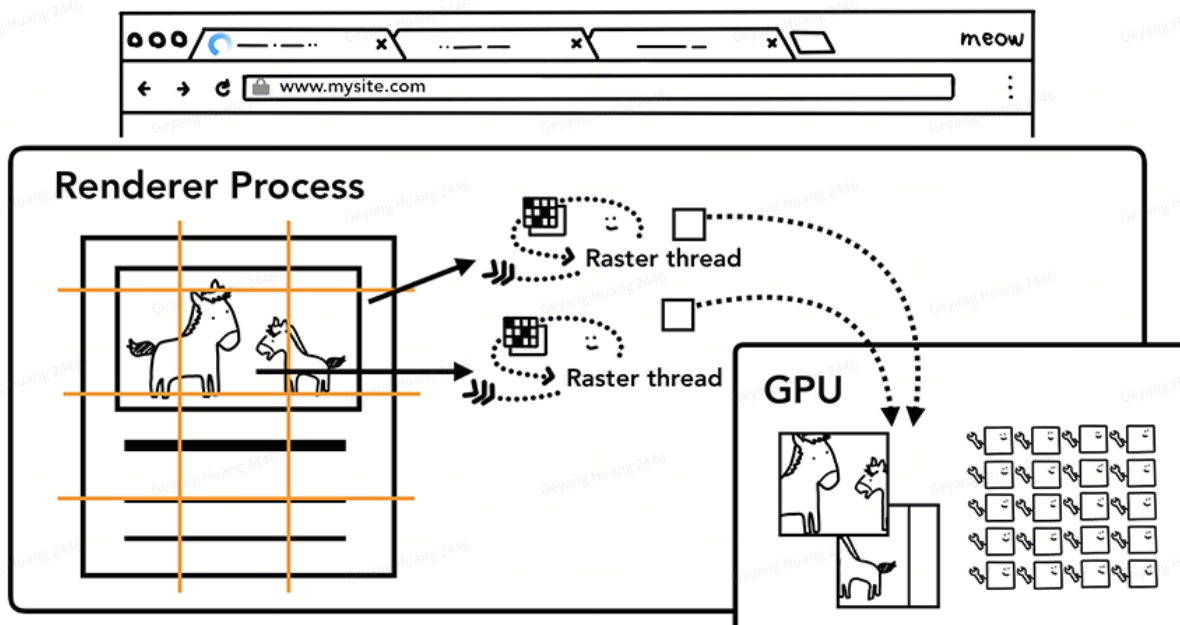


也可以使用 `will-change` CSS 属性告诉浏览器对其单独分层。你甚至可以给页面上所有的元素一个单独的层，但分层并不是越多越好，当页面的层数超过一定数量后，分层的合成操作会比在每帧中光栅化页面的一小部分还要慢。在 [Frames](#) 或 DevTools [Layers](#) 面板中可查看网站如何被分层👉。



2. 光栅化分层

光栅线程光栅化分层。一旦创建了 `Layer Tree` 并确定了绘制顺序，主线程就会向合成线程（compositor thread）提交这些信息。然后合成线程就会光栅化页面的每一层，因为页面的一层可能有整个网页那么大，所以合成器进程需要将它们切分为若干的切片然后将每个切片发送给光栅线程（raster thread），光栅线程会光栅化每个切片并且把它们存储在GPU的内存中。



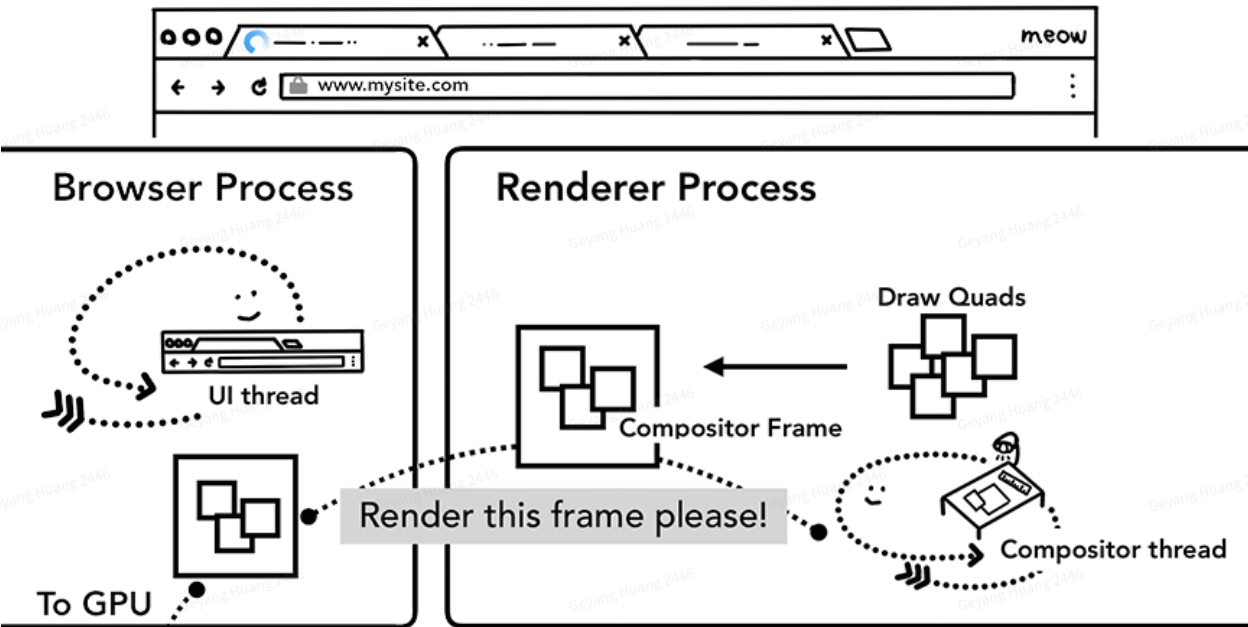
合成器进程可以对不同的光栅线程进行优先级排序，以便优先对视口内（或附近）的内容进行光栅化。

3. 合成

对切片进行光栅化后，合成器线程会收集切片信息（draw quads）来构建一个合成帧（compositor frame）。

- `draw quads`：包含切片在内存的位置以及图层合成后切片在页面的位置信息。
- `compositor frame`：绘制一帧内容的 draw quads 集合。

然后通过IPC将合成帧提交给浏览器进程（browser process）。此时 UI 线程或其他渲染器进程也可以添加另外的合成帧，这些合成帧被发送到 GPU 以显示在屏幕上。如果如果出现滚动事件，合成器线程会创建另一个合成帧发送到 GPU 用来更新页面。

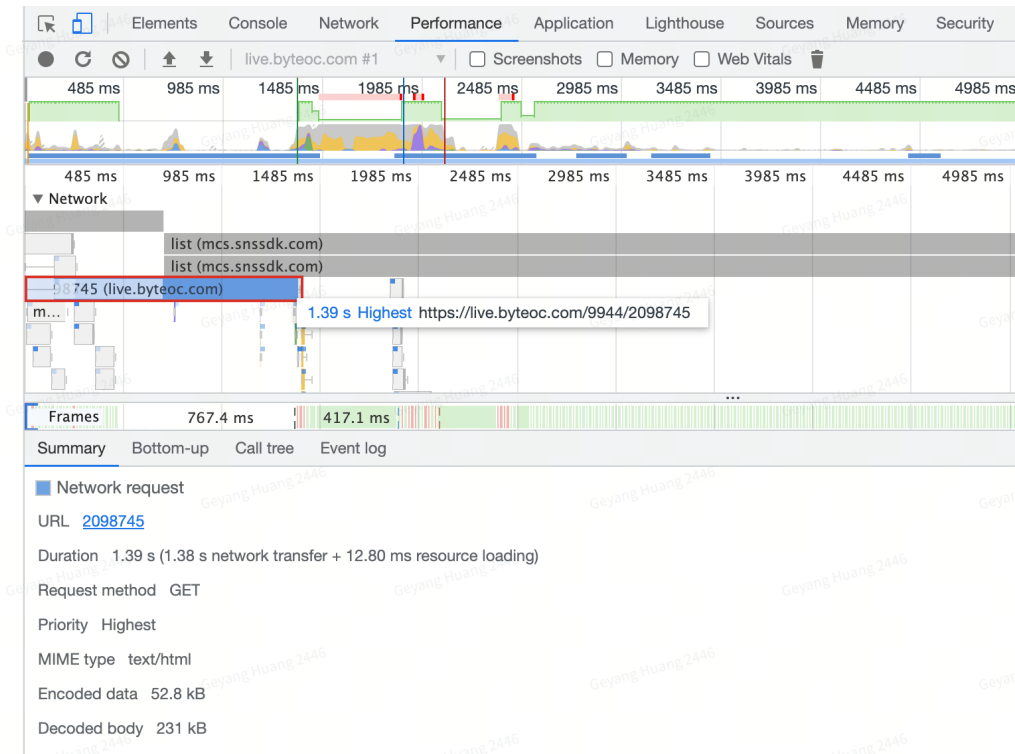


合成的好处在于它无需涉及主线程即可完成。合成器进程不需要等待样式计算（Style calculation）或 JavaScript 执行。这就是为什么[仅通过合成实现动画](#)是最推荐的方式，能取得最好的性能体验。如果需要重新计算布局（layout）或绘制（paint）则必须涉及到主线程。

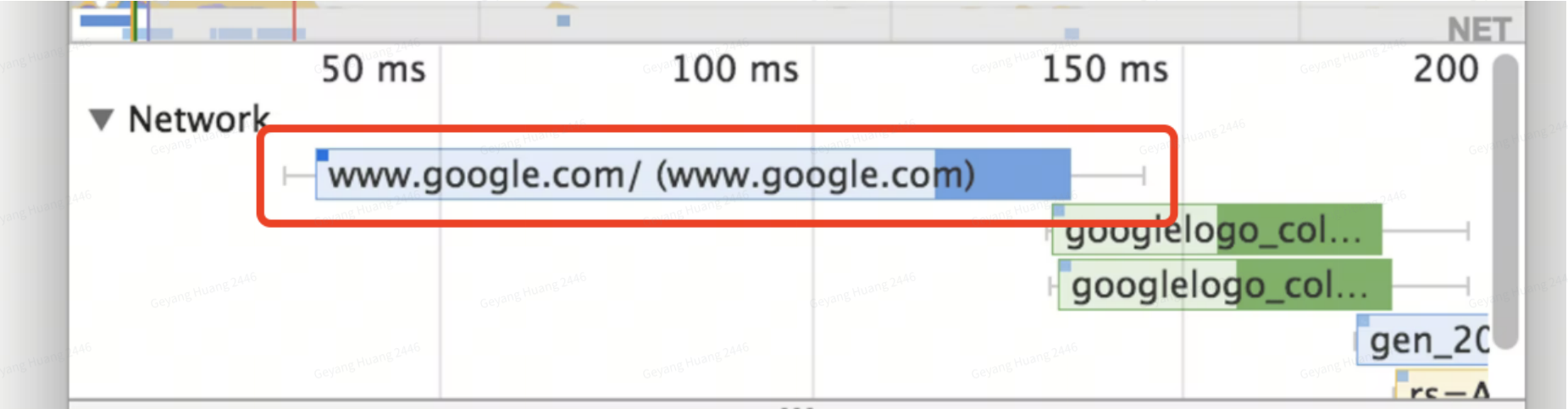
视图面板

Network

资源网络请求的瀑布流。不同的颜色用于区分不同的资源类型（HTML、JS、Image等），鼠标悬浮可查看资源请求总耗时和优先级（Highest、High、medium、Low、Lowest）。鼠标点击某个资源请求可以在 `Summary` 选项卡中查看更详细的信息。

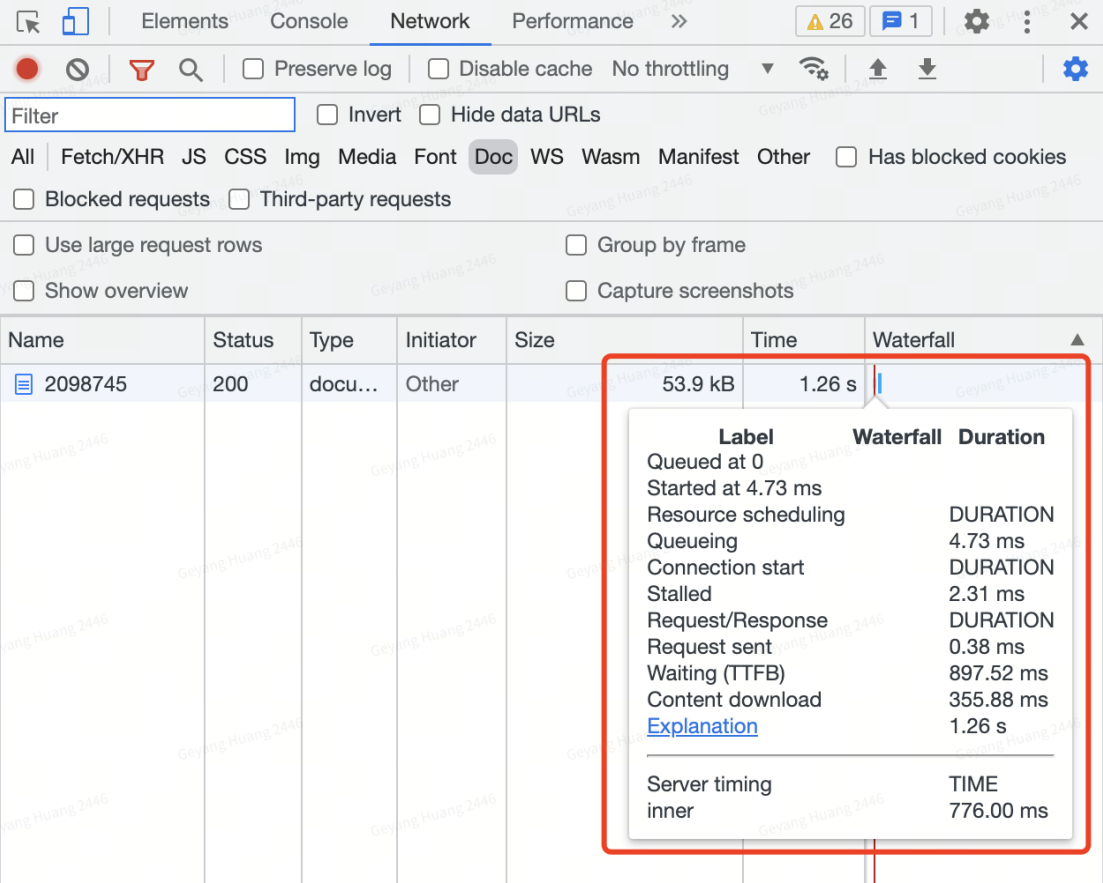


每个资源请求的图形中都存在一左侧线段、浅色矩形、深色矩形、右侧线段：



- 左侧线段：Request Sent 之前所有时间花费（Queueing ~ Proxy negotiation，后有详见👉）。
- 浅色矩形：Request Sent + Waiting (TTFB)。从请求发出到浏览器接收到服务端响应的第一个字节花费的时间。
- 深色矩形：Content Download，内容下载耗时。
- 右侧线段：等待主线程的时间花费。

如果想更具体了解该网络请求的耗时分配，可以在 Network 瀑布图中查看该请求的具体时间序列：

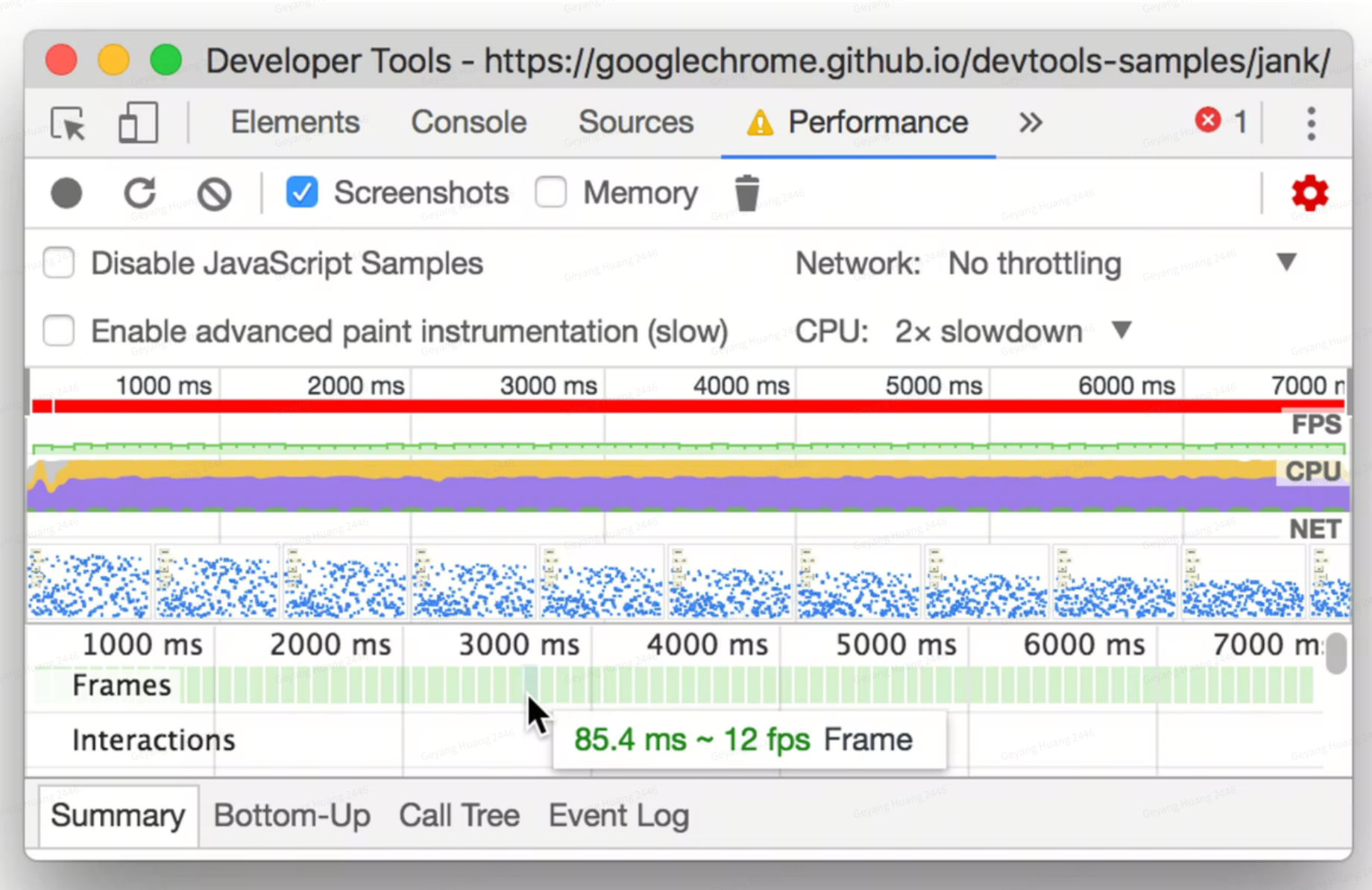


- Queueing：队列等待时间。以下情况下请求会进行排队：

- 存在更高优先级的请求；
- 同源仅能打开六个 TCP 连接的限制（仅适用于 HTTP/1.0 和 HTTP/1.1）；
- 浏览器需要在磁盘缓存中短暂分配空间。
- Stalled：请求停滞。请求可能因排队中所述的任何原因而停止。
- DNS Lookup：DNS 查找。浏览器正在解析请求的 IP 地址。
- Initial connection：初始连接。浏览器正在建立连接，包括 TCP 握手/重试和协商 SSL。
- Proxy negotiation：代理协商。浏览器正在与 proxy server 协商请求。
- Request sent：请求已发送。正在发送请求。
- ServiceWorker Preparation：ServiceWorker 准备。浏览器正在启动 Service Worker。
- Request to ServiceWorker：请求 ServiceWorker。请求正在发送给 service worker。
- Waiting (TTFB)：等待（TTFB，Time To First Byte）。浏览器正在等待响应返回的第一个字节，此时间包括 1 次往返延迟和服务端准备响应所用的时间。
- Content Download：内容下载。浏览器直接从网络或 Service Worker 接收响应。Content Download 表示读取响应内容所花费的总时间。大于预期的值可能表示网络速度较慢，或者浏览器正忙于执行其他工作，从而延迟了响应的读取。
- Receiving Push：接收推送。浏览器正在通过 HTTP/2 服务器推送接收此响应的数据。
- Reading Push：读取推送。浏览器正在读取之前接收到的本地数据。

Frames

查看每秒帧数。将鼠标悬停在其中一个绿色方块上会显示该帧的耗时和 FPS，如出现红色方块则表示出现掉帧。点选某帧可在 Summary 看到更多信息（触发该帧的相对时间和 CPU time）。



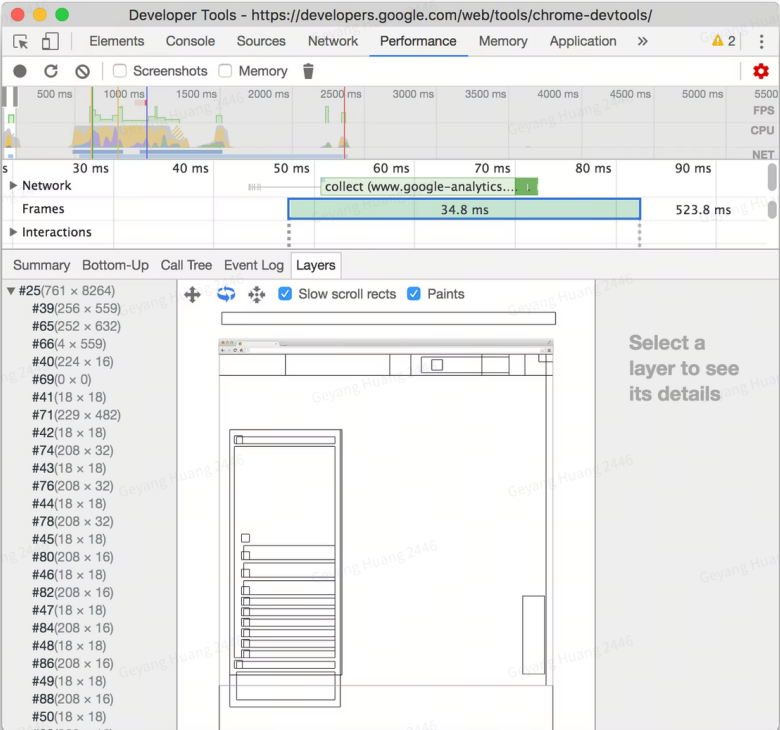
查看分层信息

开启图层信息：

- 控制面板开启 Enable advanced paint instrumentation。

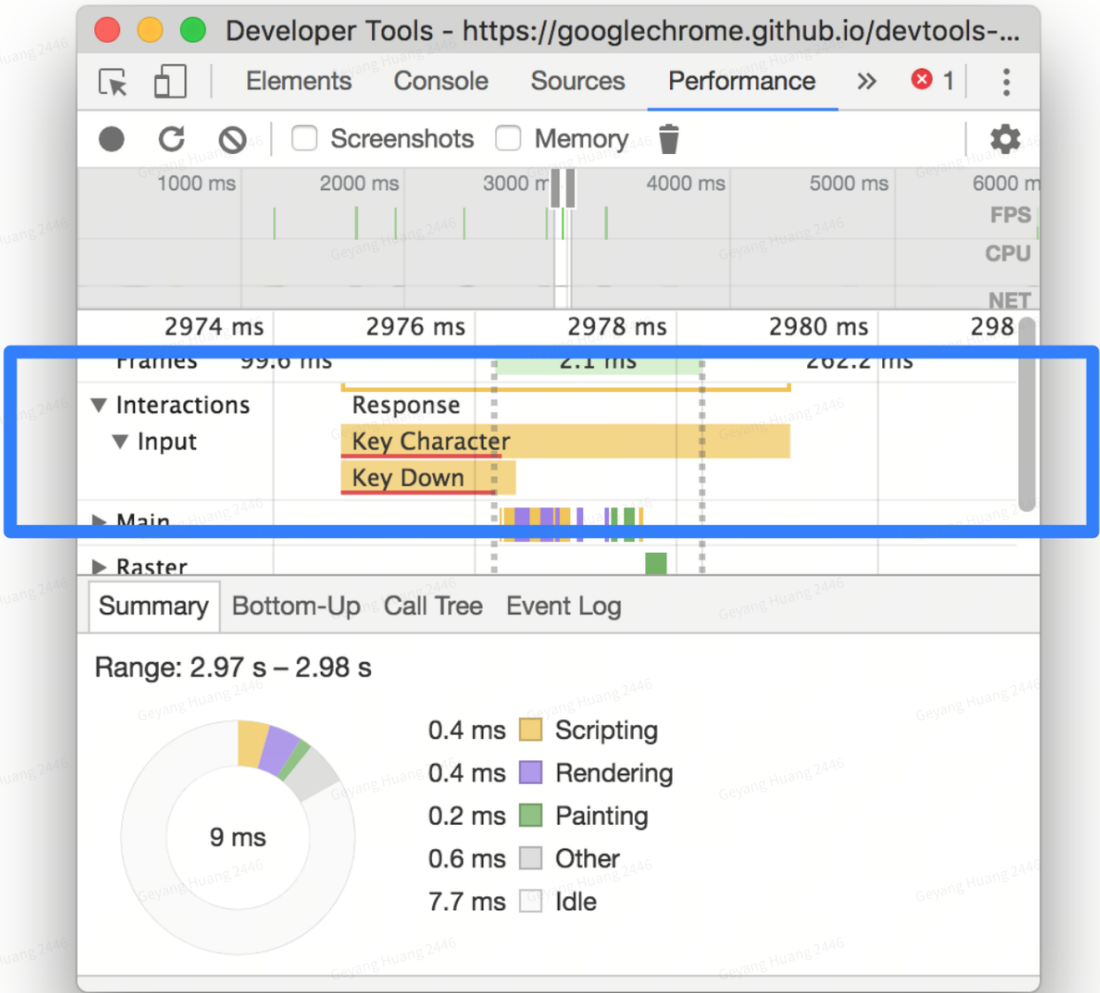
在 **Frames** 栏中选择某一帧。

分层信息将在详情面板中的 **Layers** 选项卡中展示，可查看分层结果是否符合期望：



Interactions

可查找和分析录制期间发生的用户交互。底部的红线表示等待主线程所花费的时间。

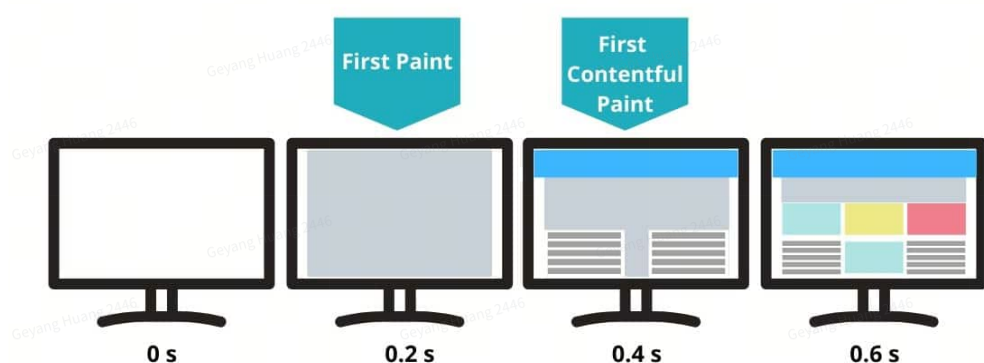


Timings

记录页面加载期间的重要阶段事件及其触发时间点。



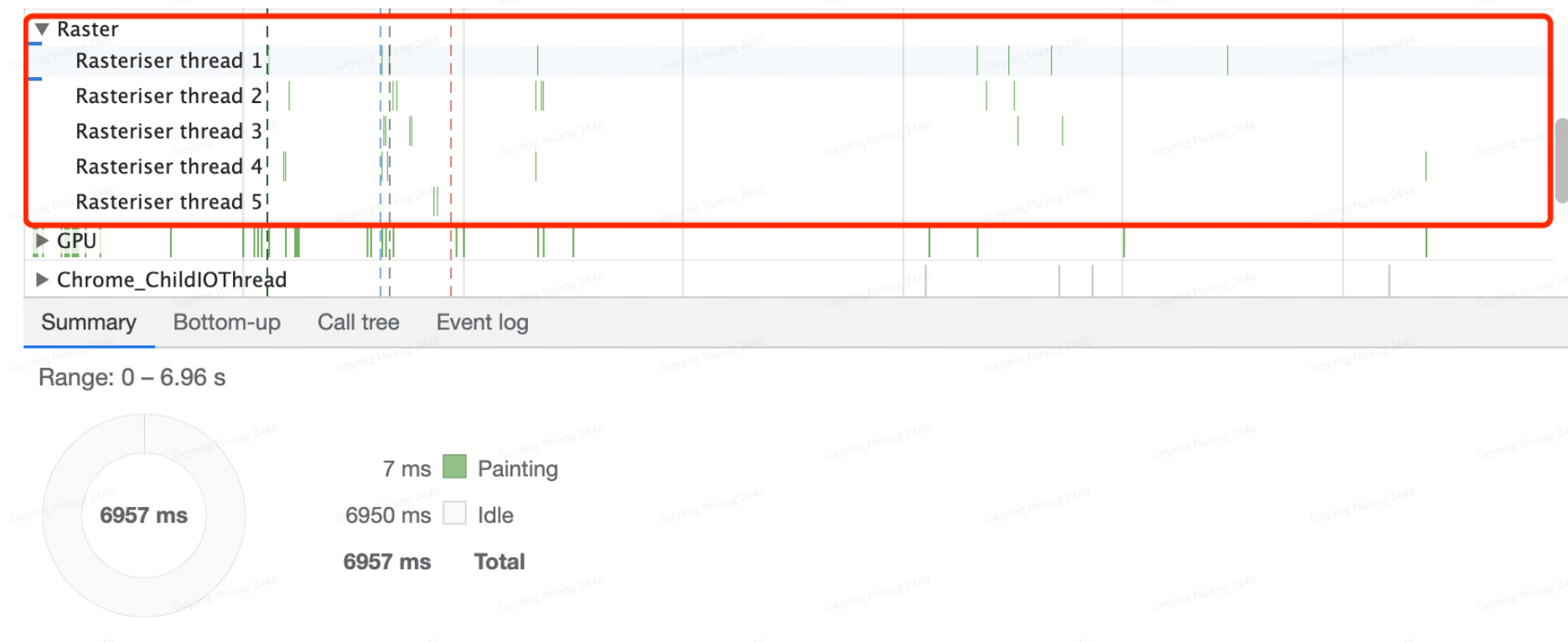
- FP (First Paint) : 页面第一个像素渲染到屏幕上所用的时间。
- FCP (First Contentful Paint) : 开始绘制内容的时间，内容包括任何文字、图片、非空白的 canvas 或 SVG 等。



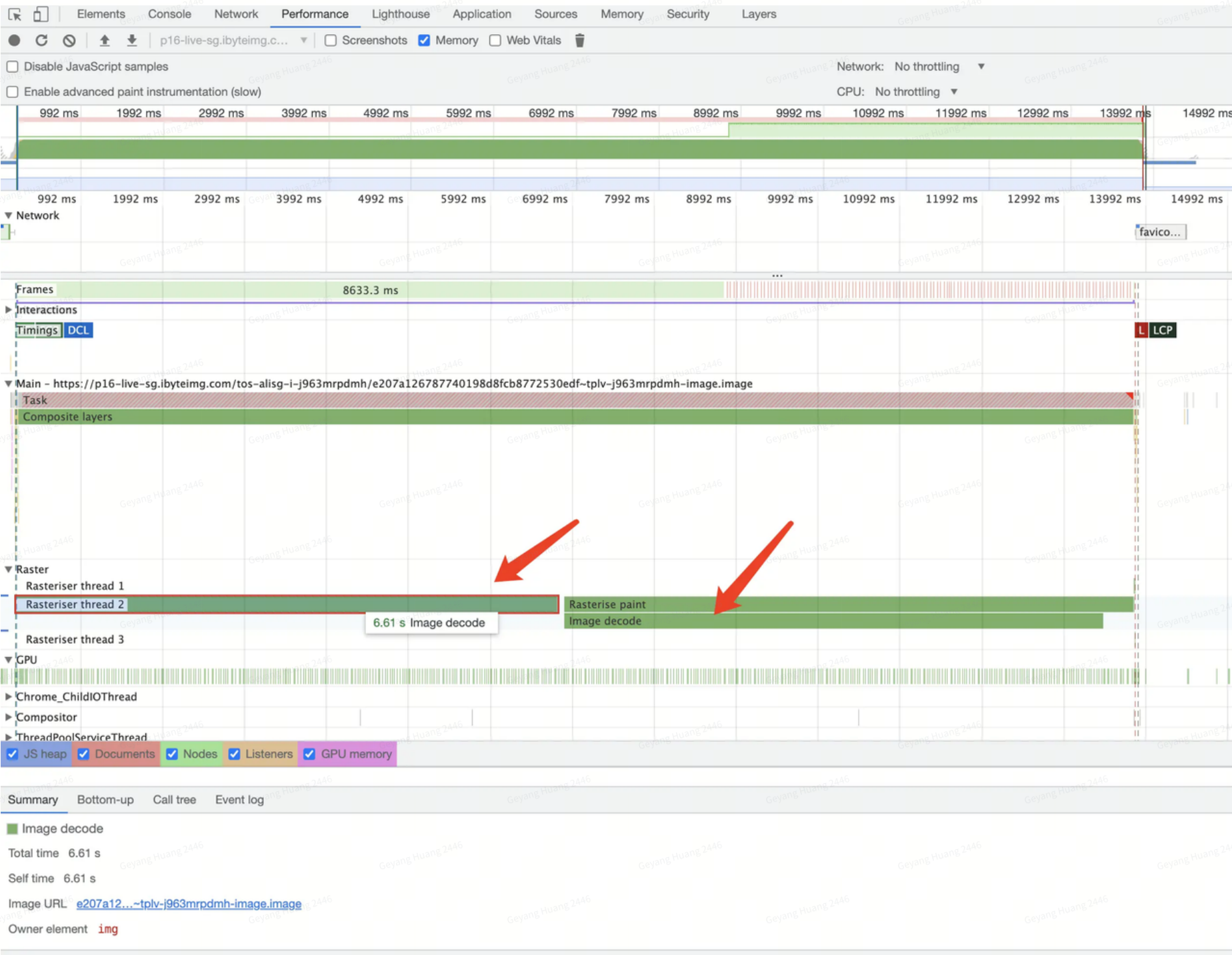
- LCP (Largest Contentful Paint) : 页面可视区内尺寸最大的元素完成渲染的时间。
- DCL (DOMContentLoaded) : 表示 HTML 已完全加载和解析，此时样式、图像、iframe 等可能尚未加载。
- L (Onload) : 页面所有资源加载完成后触发的事件。

Raster

光栅线程，它是浏览器进行光栅化（rasterizing）的进程。

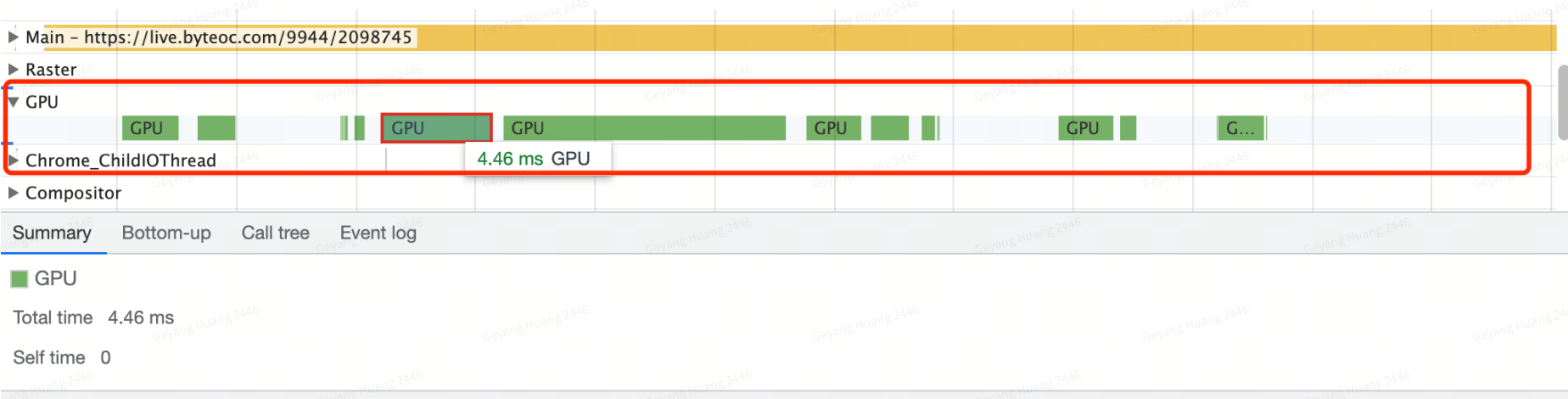


这里可以帮助我们定位渲染过程中的性能问题。比如同样是压缩后为1M左右的图片，如果它颜色比较单一，压缩后同样可以变得很小，但由于分辨率很高，浏览器解码时的CPU消耗和实际渲染内存消耗可能非常大，从而浏览器直接卡死。Network 中只能看出传输大小（压缩后），Raster 栏则可以定位出问题，如下图中可以看到主线程中有一个13s的长任务导致页面卡死，具体是一个 Composite Layers 任务（Painting类型），因此可在 Raster 中查看具体原因，定位出是两次 Image decode 耗时均在6s以上，从而导致了主线程阻塞，Summary 中可看到具体 Image URL。



GPU

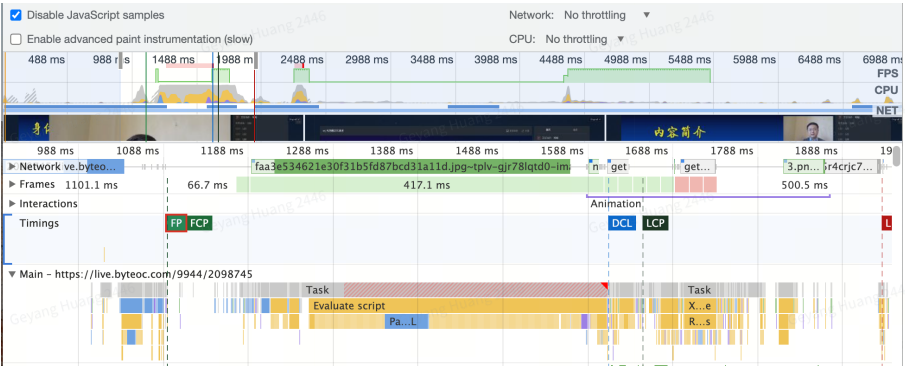
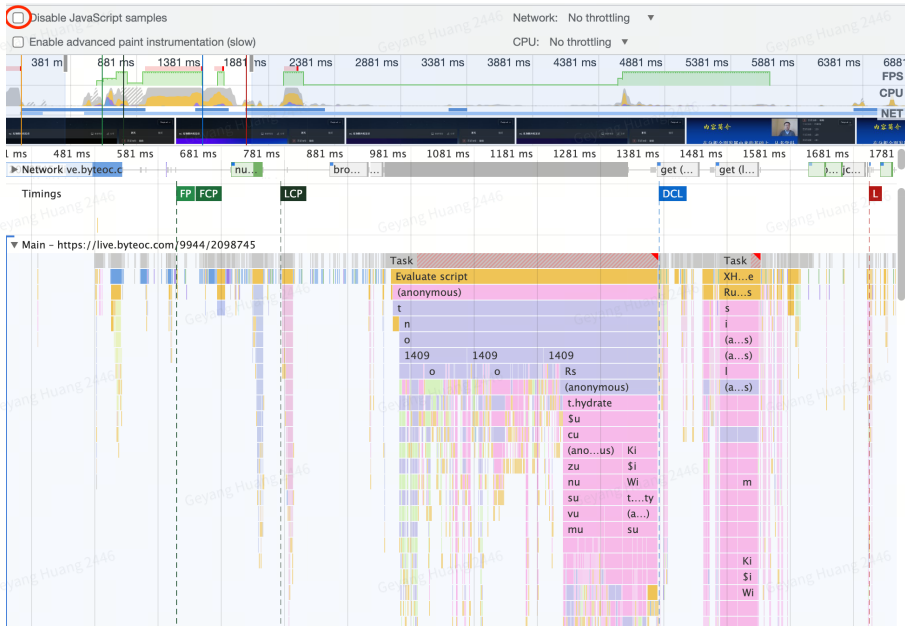
GPU 任务，鼠标悬浮可查看任务具体耗时。



Main

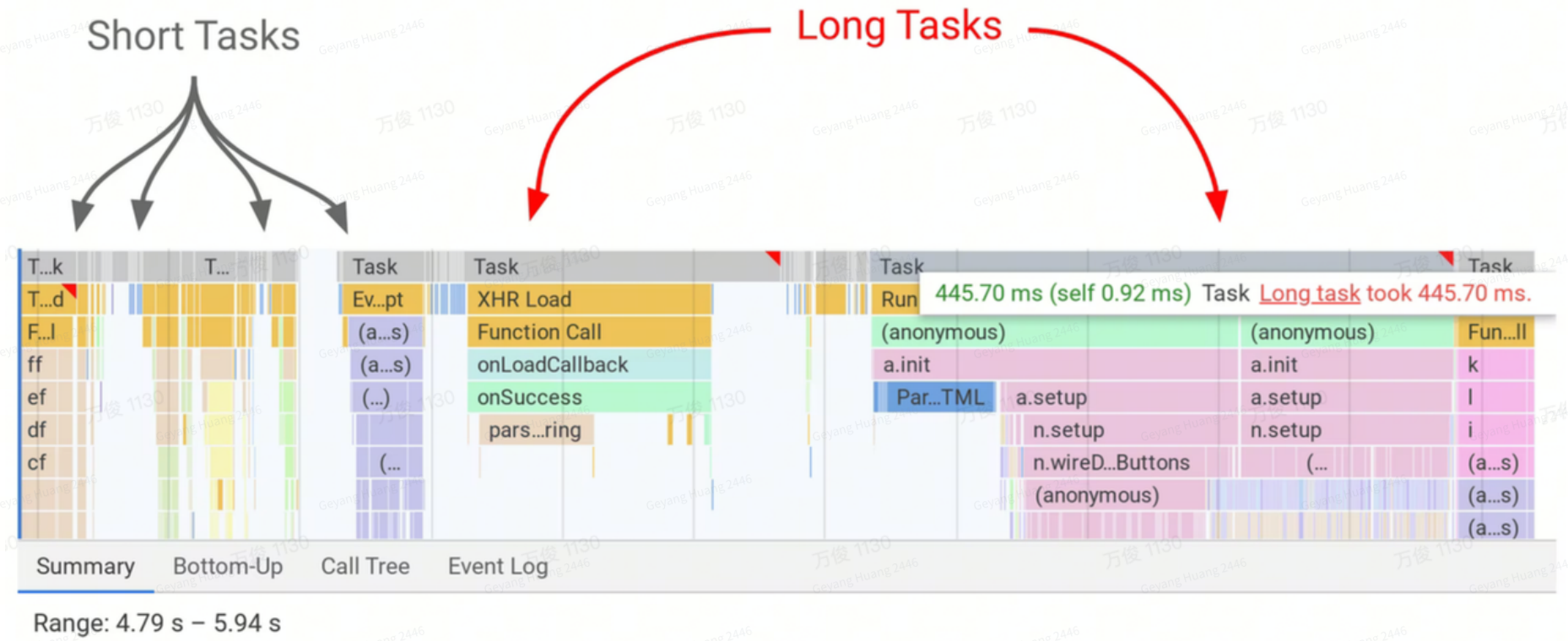
主线程活动。通过倒置的火焰图展示主线程上发生的活动，x 轴表示随时间的记录。y 轴代表调用堆栈。火焰图中顶部的事件会导致其下方的事件，火焰图顶层宽度越大就表示该活动可能存在性能问题。

默认情况下，Main 中会详细记录 JavaScript 的调用堆栈，勾选 Disable JavaScript samples 可隐藏其调用堆栈，隐藏后的火焰图明显变低（右图）。



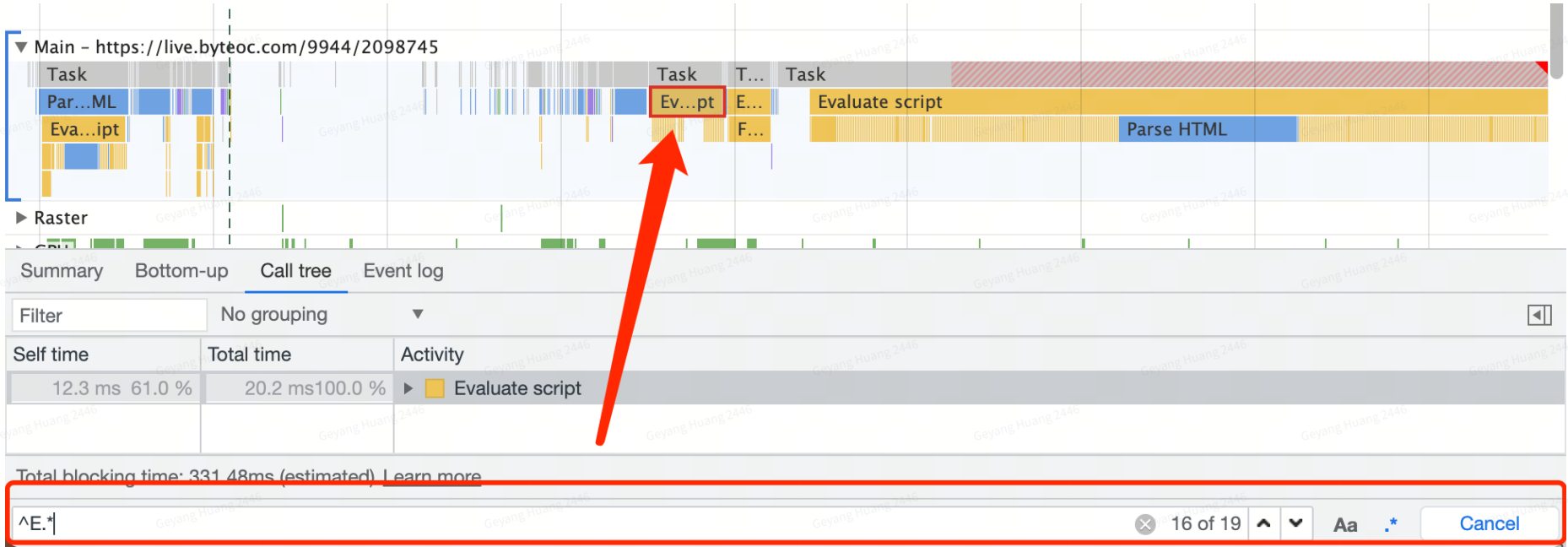
查看长任务

火焰图顶部（根部）由很多任务（Task）组成，使用灰色背景色区分。鼠标悬浮上去可以看到任务的总耗时。而超过50ms 的任务被称之为长任务，会被红色角标标记。因此 Main 视图中可查看导致掉帧的具体任务。



搜索框

按 Command+F (Mac) 或 Control+F (Windows、Linux) 打开底部的搜索框，可对火焰图中的活动进行搜索，支持正则。



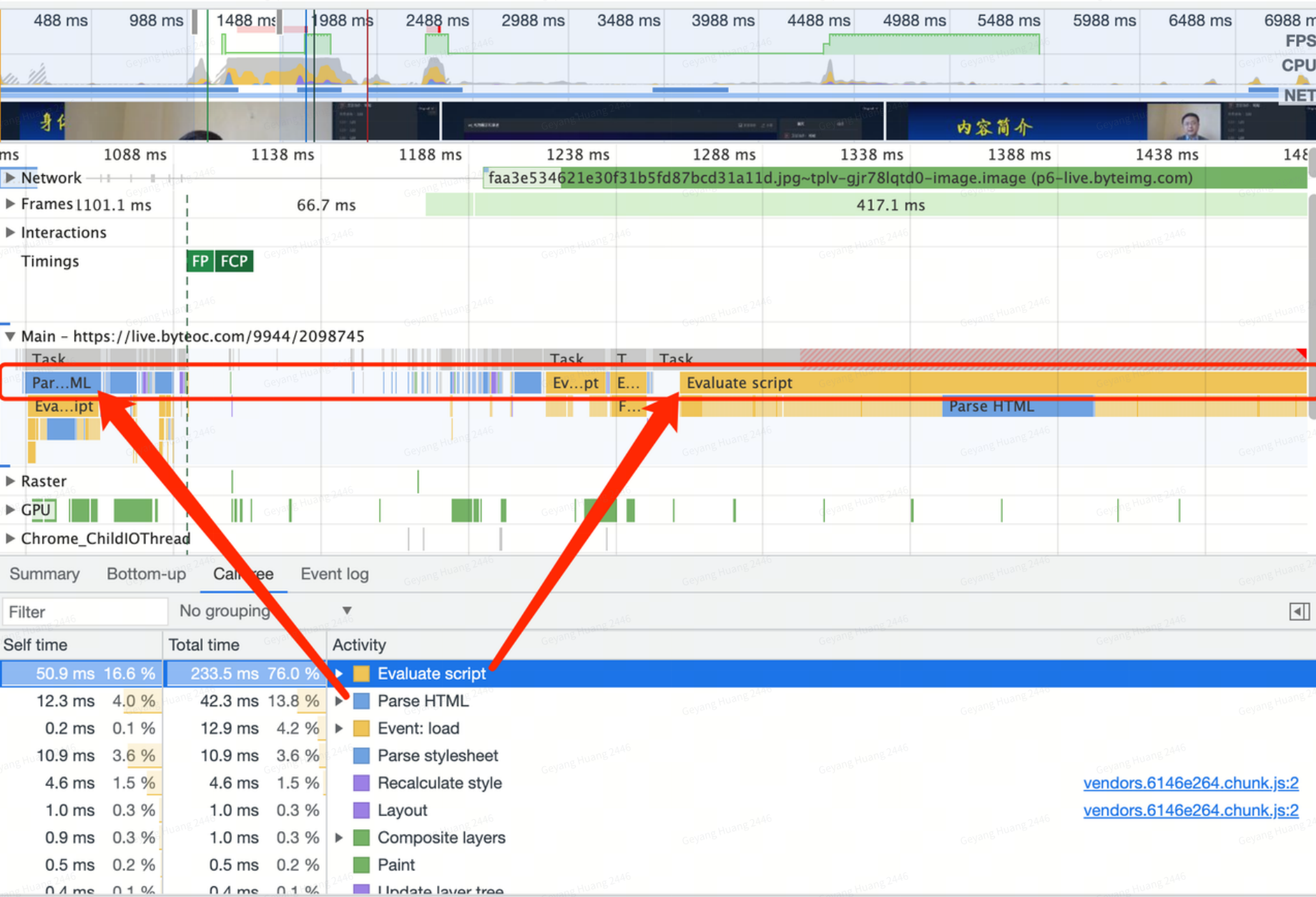
查看火焰图中的活动

点击选择火焰图的某一活动或者选择某一段时间段的活动，将在详情面板 **Summary** 选项卡中展示更多信息，结合详情面板的 **Call tree**、**Bottom-up**、**Event log** 选项卡可以进行不同维度分析。

根活动

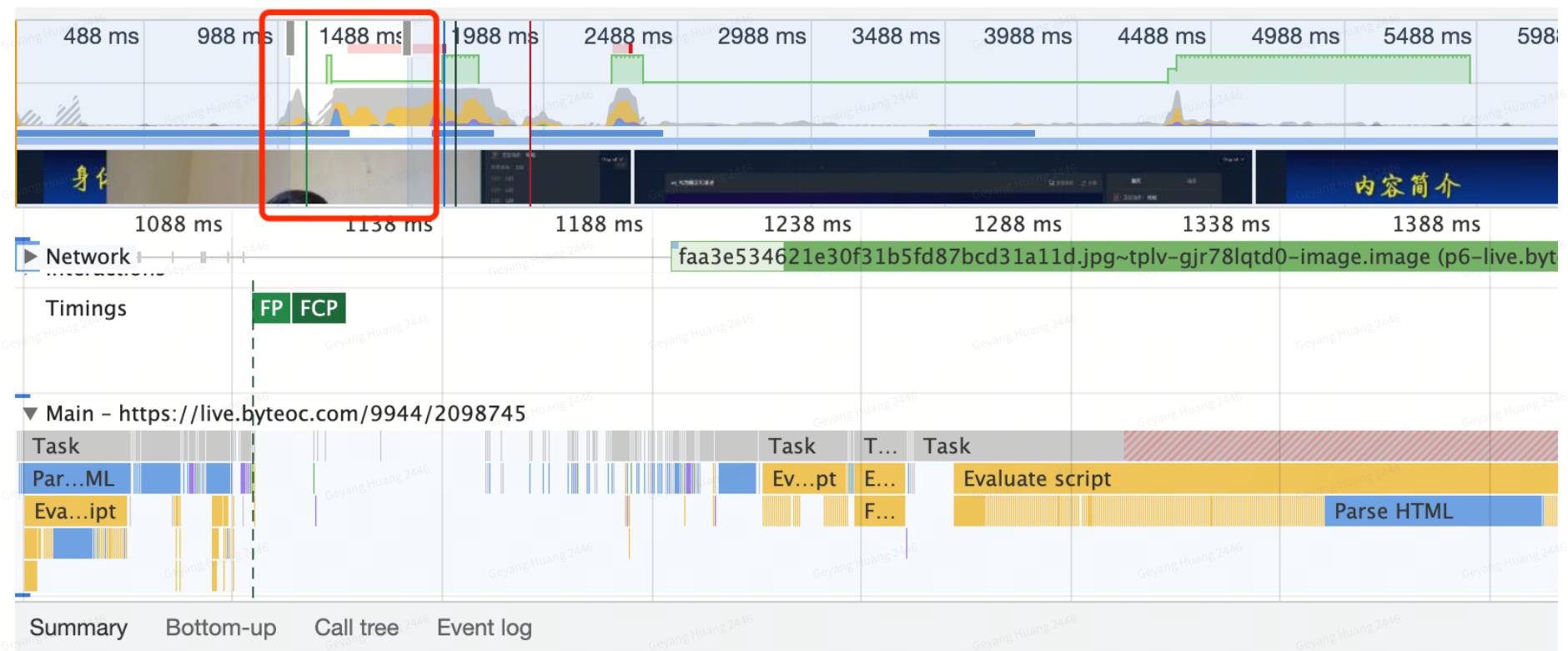
根活动是指那些导致浏览器做一些工作的活动，例如当单击一个页面时浏览器会触发一个 **Event** 作为根活动，这个 **Event** 可能会导致一系列的处理程序执行等。

根活动位于在火焰图中顶部，Task的下方，也会出现在 **Call tree**、**Event log** 选项卡 **Activity** 列的首行👉。

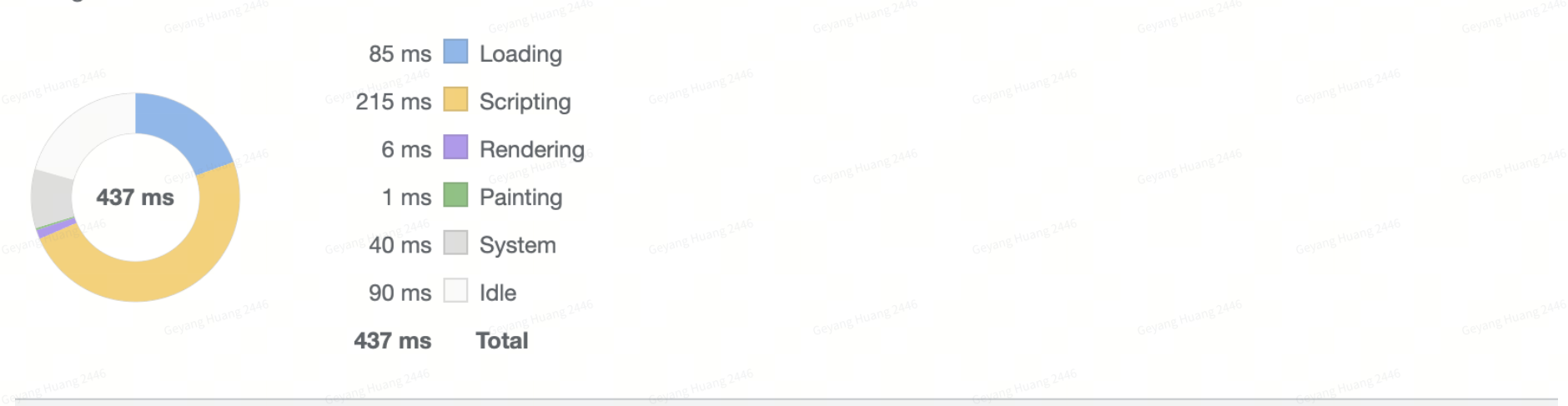


Summary

摘要，展示点选活动或某时间段所有活动的各阶段时间耗时。



Range: 1.05 s - 1.48 s



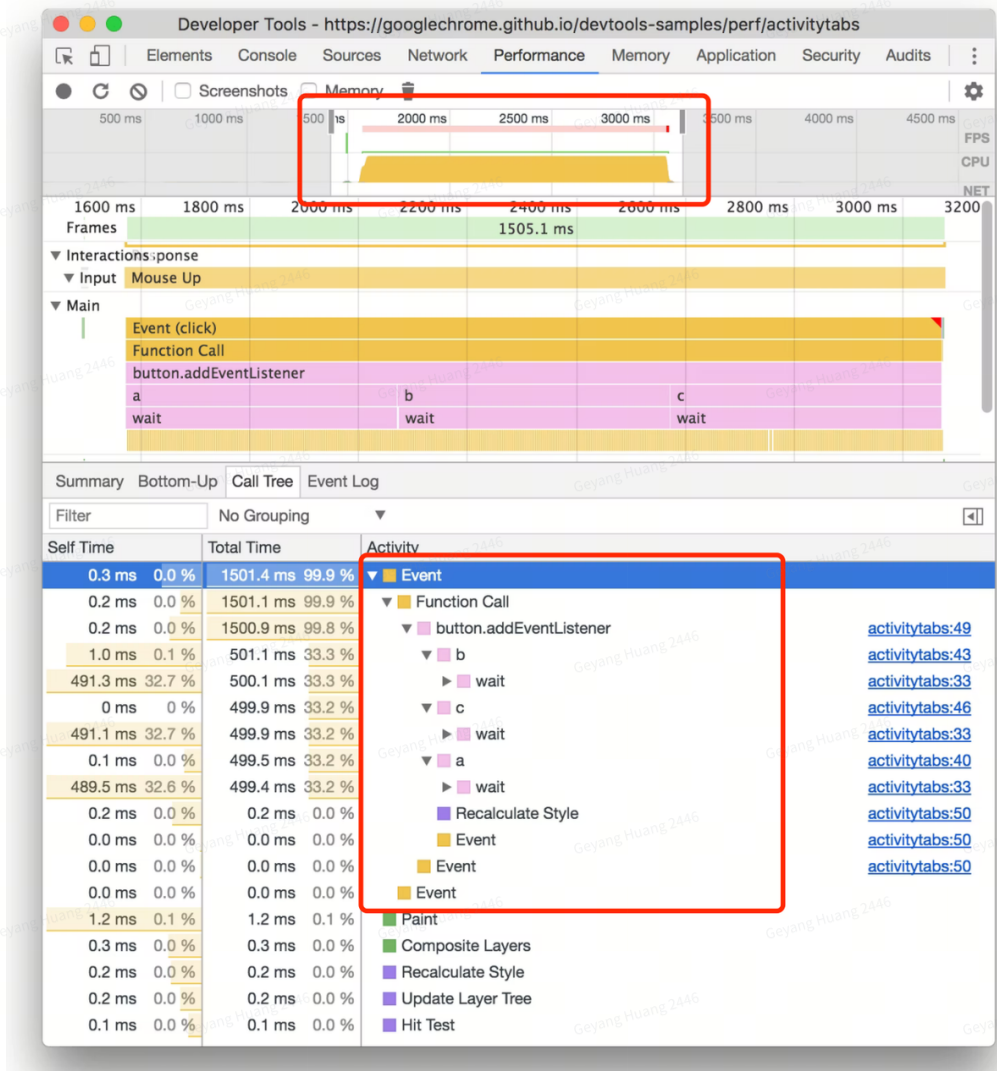
- Loading: 网络请求与解析。
- Scripting: JS 执行时间。
- Rendering: 重排，主要包含样式计算、更新布局树、布局、分层等。
- Painting: 重绘。更新分层、光栅化分层、合成等。
- System: 系统占用时间。
- Idle: 空闲时间。
- Total: 总计。

详见[这里](#)。

Call tree

调用树，通常用于查看选择的时间段中导致最多耗时的根活动。

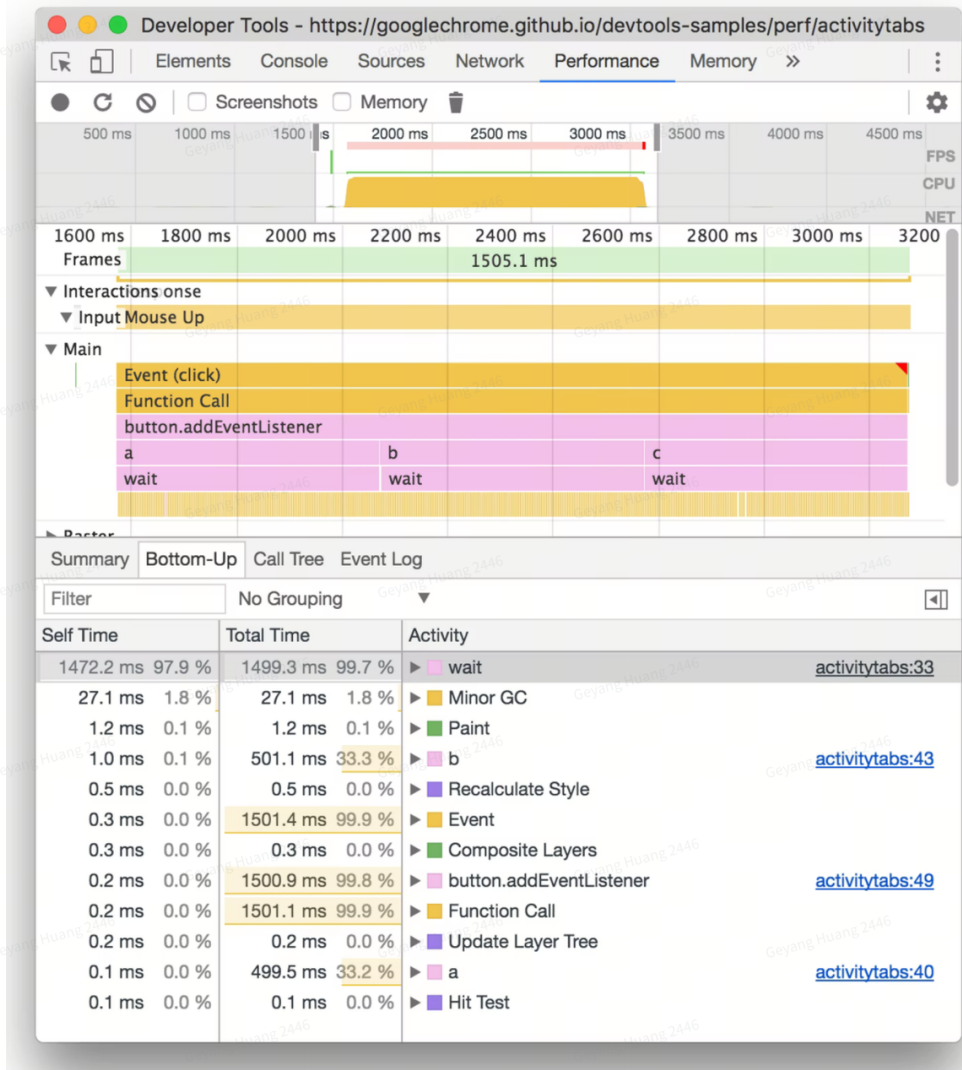
下图中 `Event` 是一个根活动，嵌套结构表示表示调用栈，表示 `Event` 导致了 `button.addEventListener`，`button.addEventListener` 中执行了 `b ...`



- Self Time：该活动直接花费的时间。
- Total Time：该活动和其所有子活动花费的时间。

Bottom-up

自下而上，通常用于查看选择的时间段中直接花费时间最多的活动。

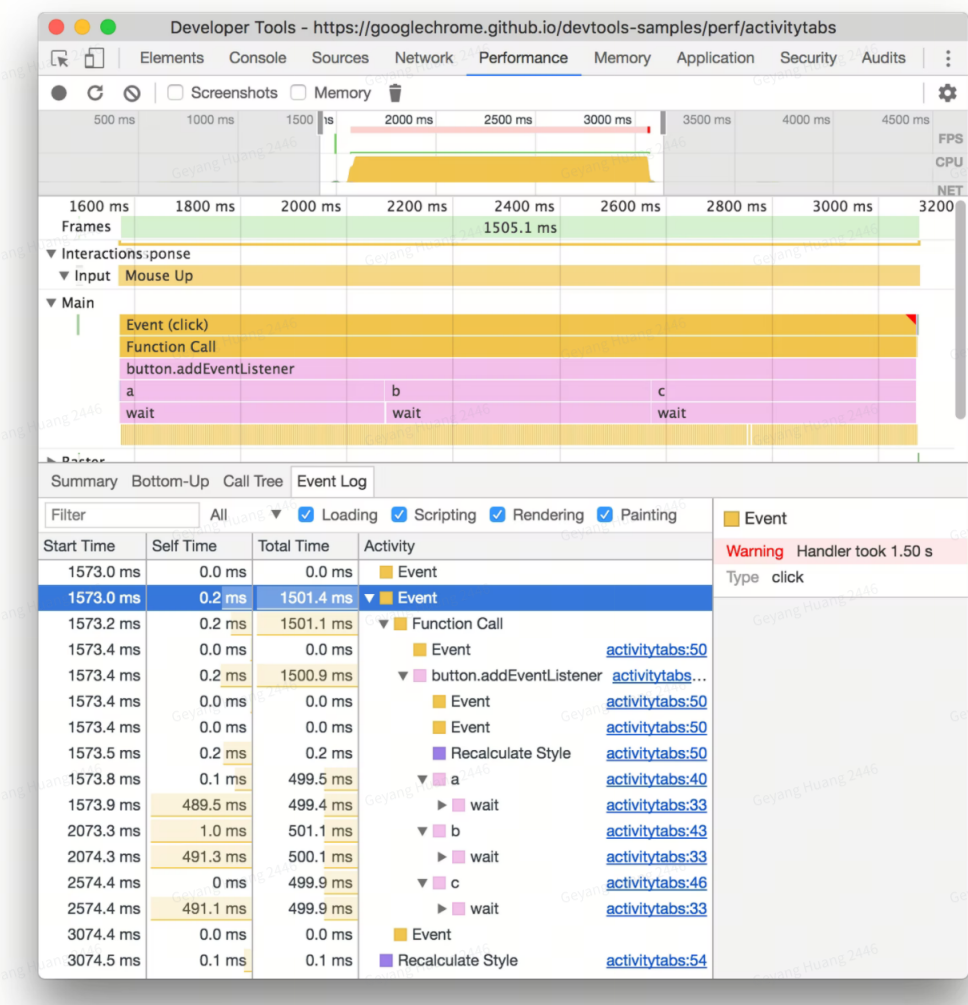


上面图例中可以看到几乎所有时间都花在了三个 `wait()` 的调用上。因此 Bottom-up 选项卡中的顶部活动是 `wait`。观察火焰图部分可以 `wait` 实际上是数千个 `Minor GC` 调用。因此你可以看到在 Bottom-up 选项卡中下一个最昂贵的活动是 `Minor GC`。

- Self Time：该活动直接花费的时间的汇总时间，因为其可能出现了多次。
- Total Time：该活动和其所有子活动花费的时间的总时间。

Event log

事件日志，用于按照活动的发生顺序查看活动。

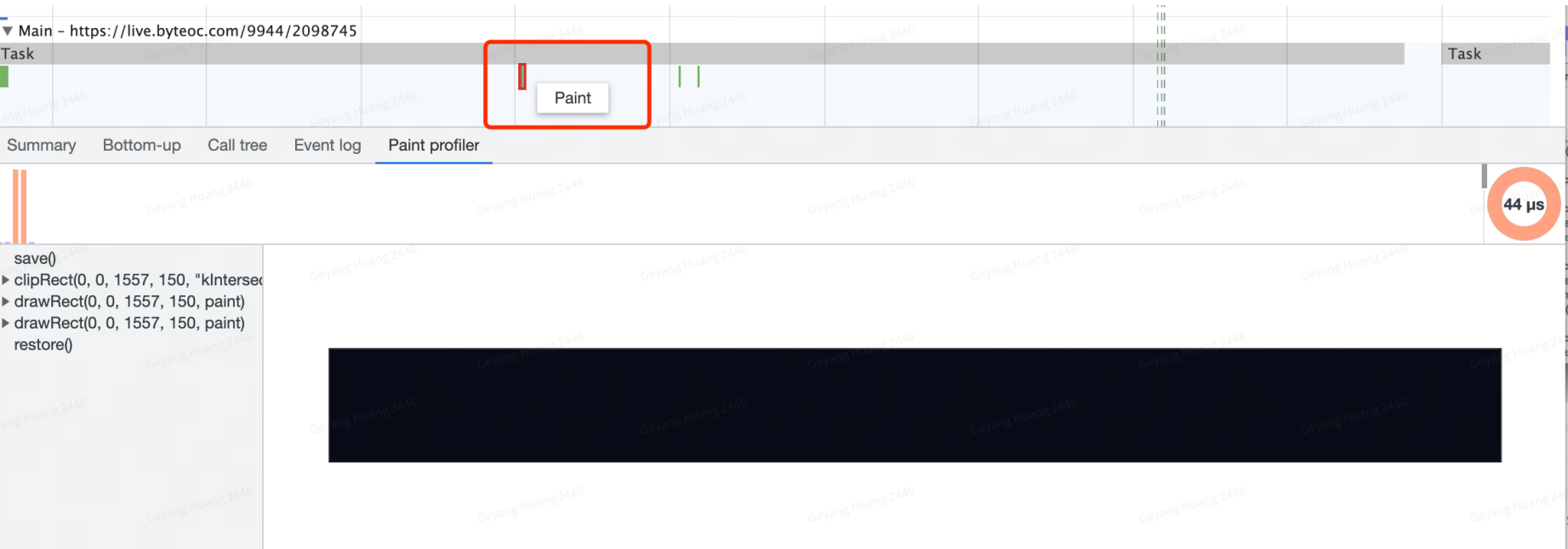


- Start Time：该活动的启动时间，它相对于录制的开始时间。
- Self Time：直接在该活动上花费的时间。
- Total Time：直接在该活动及其所有子活动上花费的时间。

Paint 分析器

开启 Paint 分析器：

- 在控制面板开启 Enable advanced paint instrumentation；
- 点击选择一个 Paint 事件。



推荐阅读

- 实战分析：[Slardar长任务归因步骤详解](#)

参考

- [Measure performance with the RAIL model](#)
- [Analyze runtime performance](#)
- [Performance features reference](#)
- [Use performance to see what the browser is doing](#)
- [Inside look at modern web browser](#)
- [Eliminate content repaints with the new Layers panel in Chrome](#)